



Università degli Studi di Salerno

Dipartimento di Ingegneria dell'Informazione ed Elettrica e Matematica
Applicata(DIEM)

Documentazione del Progetto

Sviluppo di un'applicazione Java per la gestione di una biblioteca

Corso di Ingegneria del Software – A.A. 2025/2026

Studenti Gruppo 11:

Vincenzo Galdiero

Matricola:0612709367

Gabriele Ciccone

Matricola :0612709695

Gerardo Carino

Matricola:0612710164

Donato Finiello

Matricola:0612709377



Indice

1. Introduzione

1.1 Scopo del documento.....	4
1.2 Contesto applicativo.....	4
1.3 Glossario.....	4

2. Progettazione ad alto livello

2.1 Obiettivi architetturali.....	5
2.2 Criteri di progettazione e Quality Attributes (QA).....	5
2.2.1 Attributi di qualità esterni.....	5
2.2.2 Attributi di qualità interni.....	6
2.3 Scomposizione object-oriented.....	6
2.4 Adozione del pattern MVC.....	7
2.5 Coesione e accoppiamento.....	7
2.6 Descrizione dei moduli.....	8
2.6.1 Modulo View.....	9
2.6.2 Modulo Gestione Libri.....	9
2.6.3 Modulo Gestione Utenti.....	10
2.6.4 Modulo Gestione Prestiti.....	10
2.6.5 Modulo Gestione Eccezioni.....	11
2.7 Viste architetturali del sistema.....	11
2.7.1 Vista logica.....	11
2.7.2 Vista di processo.....	11
2.7.3 Vista fisica.....	11
2.7.4 Vista di sviluppo.....	12
2.8 L'interfaccia Archivio come decisione architeturale.....	12
2.9 Diagramma dei package dettagliato.....	13

3. Progettazione di dettaglio

3.1 Modulo Gestione Libri.....	15
3.1.1 Classe Libro.....	15
3.1.2 ArchivioLibri.....	15
3.1.3 LibriService.....	15
3.1.4 LibroViewController.....	15
3.2 Modulo Gestione Utenti.....	15
3.2.1 Classe Utente.....	16
3.2.2 ArchivioUtenti.....	16
3.2.3 UtentiService.....	16
3.2.4 UtentiViewController.....	16
3.3 Modulo Gestione Prestiti.....	16
3.3.1 Classe Prestito.....	16
3.3.2 StatoPrestiti.....	17
3.3.3 ArchivioPrestitiAttivi.....	17
3.3.4 ArchivioCronologiaPrestiti.....	17

3.3.5 PrestitiService.....	18
3.3.6 PrestitiViewController.....	18
3.4 Modulo GestioneHome.....	18
3.4.1 HomeController.....	18
3.5 Modulo GestioneEccezioni.....	18
3.6 Applicazione dei principi SOLID.....	19
3.7 Qualità del software e contenimento del debito tecnico.....	20
4. Modello statico	
4.1 Diagramma delle classi - modello essenziale.....	21
4.2 Diagramma delle classi - modello completo.....	23
4.3 Descrizione dettagliata delle relazioni.....	24
4.3.1 Associazioni.....	24
4.3.2 Dipendenze.....	24
4.3.3 Aggregazioni.....	25
4.4 Analisi della coesione e dell'accoppiamento.....	25
4.4.1 Coesione.....	25
4.4.2 Accoppiamento.....	26
5. Modello dinamico	
5.1 Introduzione ai diagrammi di sequenza.....	28
5.2 Sequenza: Registrazione di un nuovo libro.....	28
5.2.1 Flusso Principale.....	28
5.2.2 Gestione dei Flussi Alternativi.....	30
5.3 Sequenza: Registrazione di un prestito.....	30
5.3.1 Flusso Principale.....	30
5.3.2 Gestione dei Flussi Alternativi.....	33
5.4 Sequenza: Restituzione di un prestito.....	34
5.4.1 Flusso Principale.....	34
5.4.2 Gestione dei Flussi Alternativi.....	36
5.5 Diagramma Macchina a Stati Finiti.....	36
5.6 Descrizione del flusso dell'applicazione.....	37
6. Design dell'interfaccia utente	
6.1 Linee guida adottate.....	39
6.2 Navigazione tra le schermate.....	39
6.3 Mock-up delle schermate.....	39
6.3.1 Home.....	39
6.3.2 Schermata Libri.....	40
6.3.3 Schermata Utenti.....	41
6.3.4 Schermata Prestiti.....	42
6.4 Gestione dei messaggi.....	43

1. Introduzione

1.1 Scopo del documento

Il presente documento ha lo scopo di descrivere in maniera organica la progettazione dell'applicazione Java sviluppata per la gestione di una biblioteca universitaria.

Vengono presentate:

- le scelte architetture adottate,
- la suddivisione in moduli,
- il modello statico (classi e relazioni),
- il modello dinamico (sequenze e attività),
- il design dell'interfaccia utente.

1.2 Contesto applicativo

L'applicazione è pensata per supportare le principali attività di una biblioteca universitaria:

- registrazione e gestione del catalogo dei libri;
- registrazione e gestione degli utenti;
- apertura e chiusura dei prestiti;
- rispetto dei vincoli di dominio (numero massimo di prestiti, copie disponibili, ecc.);
- persistenza dei dati tramite file.

La soluzione è stata realizzata utilizzando Java per la logica applicativa e JavaFX per l'interfaccia grafica.

1.3 Glossario

- Dominio: insieme dei concetti che descrivono biblioteca, utenti e prestiti.
- Archivio: componente che gestisce collezioni di oggetti e la loro persistenza su file.
- Service: strato che contiene le regole di business e la logica applicativa.
- Controller: classe che gestisce gli eventi dell'interfaccia utente.
- View: interfaccia grafica realizzata in FXML (JavaFX).

2. Progettazione ad alto livello

2.1 Obiettivi architetturali

Per il sistema di gestione della biblioteca è stata adottata un'architettura basata su Java e JavaFX, organizzata secondo il pattern Model–View–Controller (MVC).

Gli obiettivi principali della progettazione architetturale sono stati:

- garantire una netta separazione delle responsabilità;
- ridurre l'accoppiamento tra componenti;
- modellare il dominio in modo fedele e coerente (libri, utenti, prestiti, stati del prestito);
- avere una struttura chiara che faciliti la lettura del codice e l'individuazione dei ruoli di ogni classe.

2.2 Criteri di progettazione e Quality Attributes (QA)

La progettazione del sistema non si limita a soddisfare i requisiti funzionali, ma tiene conto anche delle caratteristiche di qualità che influenzano il comportamento complessivo dell'applicazione.

Nella fase architetturale è stato quindi necessario valutare quali attributi fossero più rilevanti per un sistema di gestione bibliotecaria.

L'obiettivo principale è stato individuare un equilibrio tra facilità d'uso, correttezza del funzionamento e capacità del sistema di adattarsi a modifiche future. Le scelte fatte nella scomposizione dei moduli, nell'organizzazione delle classi e nella definizione delle interfacce derivano direttamente da tali attributi di qualità.

2.2.1 Attributi di qualità esterni

Gli attributi di qualità esterni riguardano le caratteristiche osservabili da chi utilizza il sistema. Nel caso dell'applicazione sviluppata, sono stati considerati prioritari:

- Usabilità

L'interfaccia è stata progettata in modo semplice e intuitivo, così da permettere al bibliotecario di svolgere rapidamente le operazioni più comuni.

L'uso di JavaFX e di schermate strutturate a tabella facilita la consultazione dei dati e riduce la possibilità di errore.

- Efficienza

Le operazioni principali, come la ricerca e la visualizzazione dei cataloghi, devono essere rapide. La scelta di strutture dati appropriate contribuisce a mantenere i tempi di risposta costanti.

- Sicurezza dei dati

Pur trattandosi di un'applicazione desktop, sono state introdotte regole di validazione che impediscono l'inserimento di dati incoerenti o incompleti, proteggendo così l'integrità del catalogo e delle registrazioni degli utenti.

2.2.2 Attributi di qualità interni

Gli attributi interni riguardano la struttura del codice e la sua organizzazione

- Manutenibilità

Il sistema è suddiviso in moduli fortemente coesi, ciascuno dedicato a un insieme limitato e ben definito di responsabilità. Questo consente di localizzare facilmente le parti del codice da modificare e di intervenire senza rischiare effetti collaterali sugli altri moduli.

- Modularità

La separazione tra entità di dominio, archivi, servizi e controllori riduce l'accoppiamento e permette di ampliare o sostituire una componente senza impattare sull'intero progetto.

- Riusabilità

L'architettura adottata, basata su dominio, archivi, service e controller, può essere riutilizzata integralmente per sistemi analoghi, ad esempio per gestire il noleggio di dispositivi o materiali multimediali, cambiando solo le entità del dominio.

- Testabilità

La chiara separazione tra logica applicativa e interfaccia grafica rende possibile testare i servizi e gli archivi in modo isolato.

- Portabilità

La scelta di Java permette di eseguire l'applicazione su sistemi diversi senza modifiche, caratteristica utile nel contesto universitario.

2.3 Scomposizione object-oriented

La progettazione del sistema è stata guidata dall'obiettivo di ottenere una struttura chiara e coerente con il dominio applicativo della biblioteca. Tra i diversi approcci possibili, è stato adottato un modello di progettazione object-oriented, ritenuto il più adeguato per rappresentare entità reali come libri, utenti e prestiti.

In fase preliminare sono stati valutati i due approcci canonici alla scomposizione del sistema:

- scomposizione funzionale, che frammenta il software in procedure e operazioni elementari da eseguire in sequenza;
- scomposizione orientata agli oggetti, che organizza il sistema attorno a entità dotate sia di dati sia di comportamenti.

La prima modalità risulta efficace per sistemi puramente procedurali, ma tende a generare strutture difficili da estendere e con bassa coesione. Nel contesto della gestione bibliotecaria, al contrario, il problema è naturalmente modellabile attraverso oggetti che hanno un'identità e uno stato: ad esempio un libro con il suo ISBN, un utente con la sua matricola o un prestito con il proprio ciclo di vita.

Per questo motivo l'approccio orientato agli oggetti si è rivelato più adatto: consente di associare i dati e le operazioni che li riguardano all'interno della stessa classe, migliorando la chiarezza dei ruoli e la coesione interna dei moduli. Inoltre, la progettazione a oggetti facilita il controllo dell'accoppiamento, poiché le interazioni fra le componenti avvengono attraverso metodi e interfacce ben definite, evitando dipendenze non necessarie.

2.4 Adozione del pattern MVC

Nella progettazione del sistema è stato adottato il pattern Model–View–Controller (MVC), una soluzione architetturale che consente di separare in modo netto i dati, la logica applicativa e l'interfaccia utente. Tale scelta risponde direttamente alle esigenze di qualità identificate nella fase architetturale.

L'adozione del pattern nasce dalla necessità di evitare che la gestione grafica e la logica del dominio si influenzino reciprocamente. In un sistema come quello della biblioteca, infatti, le operazioni legate al catalogo, agli utenti e ai prestiti devono poter evolvere nel tempo senza richiedere modifiche all'interfaccia grafica, e viceversa. La separazione proposta da MVC permette di ottenere questo risultato attraverso tre componenti con ruoli distinti:

- **Model:** rappresenta lo strato più vicino al dominio applicativo. Include le entità (Libro, Utente, Prestito), gli archivi per la gestione dei dati e servizi. Lo stato del sistema e i vincoli di dominio risiedono interamente in questa parte.
- **View:** comprende le interfacce grafiche realizzate con JavaFX e definite tramite file FXML. Le view mostrano all'utente l'informazione elaborata dal sistema, ma non applicano alcuna logica né manipolano direttamente i dati.
- **Controller:** funge da ponte tra vista e modello. Intercetta gli input provenienti dall'interfaccia grafica, richiama i servizi appropriati e aggiorna la view in base allo stato corrente del sistema.

2.5 Coesione e accoppiamento

La qualità strutturale del sistema dipende dal modo in cui le sue parti sono state suddivise e collegate tra loro. Per progettare un software facilmente modificabile e stabile nel tempo, sono stati seguiti due principi fondamentali: alta coesione all'interno dei moduli e basso accoppiamento tra componenti diversi.

Coesione

La coesione indica quanto le responsabilità presenti in un modulo siano legate tra loro.

Nel progetto si è puntato a moduli coesi, ciascuno dedicato a un compito specifico del dominio:

- il modulo dei libri gestisce il catalogo e le sue regole;
- quello degli utenti si occupa dell'anagrafica e dei controlli associati;
- il modulo prestiti governa il ciclo di vita dei prestiti;
- le eccezioni sono riunite in un package unico.

Questa organizzazione rende ogni parte autonoma e facilmente comprensibile e rispecchia una coesione funzionale, il livello più elevato, perché ogni modulo realizza un unico compito ben definito.

Accoppiamento

L'accoppiamento misura quanto un modulo dipende da un altro.

Per ridurre interdipendenze inutili, la comunicazione tra componenti avviene solo attraverso metodi pubblici e non tramite accesso diretto alle strutture interne.

Gli archivi incapsulano i propri dati, i servizi mediano tra controller e modello, e l'interfaccia Archivio limita ulteriormente le dipendenze tra le varie parti.

Il risultato è un sistema che utilizza un accoppiamento per dati, uno dei livelli più desiderabili, evitando casi come variabili globali o strutture condivise.

2.6 Descrizione dei moduli

Per garantire ordine e manutenibilità, il sistema è stato organizzato in moduli funzionalmente distinti.

La Figura 1 mostra la vista statica dell'architettura attraverso il Diagramma dei Package, in cui il package principale biblioteca è suddiviso in sottopackage specializzati.

Il package `gestionelibri` comprende le classi dedicate alla gestione del catalogo librario e alle operazioni sui libri.

Il package `gestionementi` raccoglie le componenti responsabili dell'anagrafica degli utenti e delle relative validazioni.

Il package `gestioneprstiti` rappresenta il nucleo applicativo del sistema, poiché coordina il ciclo di vita dei prestiti e dipende funzionalmente sia dalla gestione degli utenti sia da quella dei libri.

Il package `gestionehome` contiene gli elementi legati alla schermata iniziale e alla navigazione dell'applicazione.

Il package `gestioneeccezioni` fornisce un supporto trasversale, centralizzando la gestione delle eccezioni personalizzate utilizzate dai vari moduli applicativi.

Il package *view*, esterno a *biblioteca*, è dedicato all'interfaccia grafica JavaFX e comunica esclusivamente con i moduli applicativi, senza accedere direttamente alla logica interna. Le dipendenze rappresentate nel diagramma indicano un uso dei servizi tra i moduli, mantenendo un basso accoppiamento e una chiara separazione delle responsabilità.

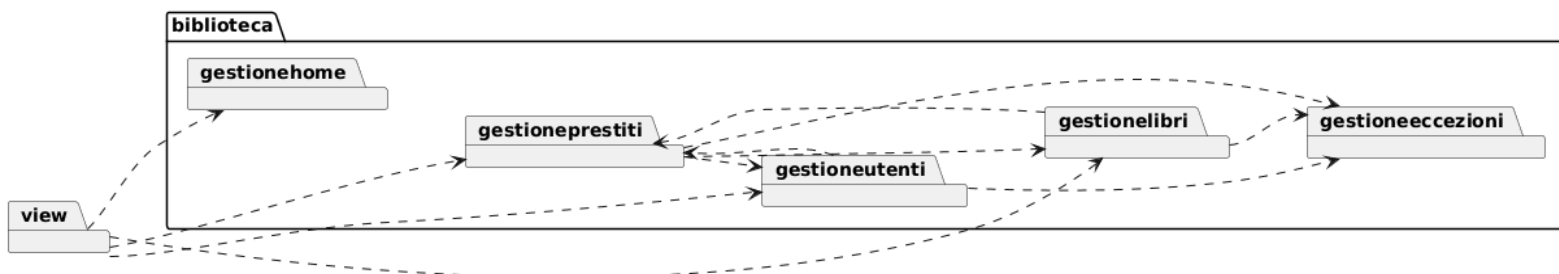


Figura 1 – Diagramma dei package dell'applicazione

2.6.1 Modulo view

Questo modulo raccoglie tutte le schermate con cui il bibliotecario interagisce durante l'uso dell'applicazione.

L'interfaccia è realizzata in JavaFX, definita tramite file FXML progettati con l'ausilio di SceneBuilder, così da separare in modo netto la parte grafica dal codice Java.

All'interno di questo package vengono gestiti:

- la visualizzazione dei dati in tabelle, etichette (label) e campi di input;
- la raccolta degli input inseriti dall'utente (click, testo, selezioni);
- l'invio di questi input ai controller corrispondenti;
- la presentazione di messaggi informativi, di errore o di conferma.

In questo modo la view si occupa esclusivamente di “come” mostrare le informazioni, lasciando tutta la logica di calcolo e controllo agli altri livelli dell'applicazione.

2.6.2 Modulo gestionelibri

Il modulo di gestione libri ha il compito di amministrare il catalogo della biblioteca.

Non si limita a memorizzare o cancellare i libri, ma svolge un vero e proprio ruolo di controllo e tutela dell'integrità dei dati.

Quando viene inserito un nuovo libro, il modulo verifica che:

- i dati inseriti siano completi e coerenti;
- non esistano duplicati rispetto ai libri già presenti.

Allo stesso modo, prima di rimuovere un libro dal catalogo, il sistema controlla che non sia ancora legato a prestiti attivi, evitando così situazioni incoerenti.

All'interno del modulo sono presenti:

- la classe Libro, che rappresenta il singolo volume;
- ArchivioLibri, che mantiene la collezione dei libri gestiti dal sistema;
- LibriService, che applica le regole di dominio (validazioni, controlli, vincoli);
- LibroViewController, che mette in comunicazione la view con il servizio.

2.6.3 Modulo gestione utenti

Questo modulo si occupa della gestione degli utenti iscritti alla biblioteca, curandone l'anagrafica e il rispetto delle regole a loro collegate. Il funzionamento è analogo a quello del modulo libri, ma applicato alle persone registrate nel sistema.

In particolare, il modulo:

- controlla che la matricola rispetti il formato previsto ed sia univoca;
- verifica che tutti i campi obbligatori (nome, cognome, e-mail, ecc.) siano compilati;
- impedisce l'eliminazione di un utente che risulta ancora titolare di uno o più prestiti attivi.

Le classi principali del modulo sono:

- Utente, che rappresenta il singolo iscritto;
- ArchivioUtenti, che conserva e gestisce l'insieme degli utenti;
- UtentiService, che si occupa di validazione, inserimenti, modifiche e cancellazioni;
- UtentiViewController, che mette in comunicazione la view per realizzare l'interfaccia grafica dedicata alla gestione utenti.

2.6.4 Modulo gestione prestiti

Il modulo di gestione prestiti rappresenta il "cuore" funzionale dell'applicazione, perché mette in relazione i libri con gli utenti e segue l'intero ciclo di vita del prestito.

A partire dalla richiesta iniziale fino alla restituzione, il modulo:

- verifica se l'utente può aprire un nuovo prestito, controllando il limite massimo di tre prestiti attivi;
- controlla che il libro richiesto abbia ancora copie disponibili;
- aggiorna automaticamente il numero di copie disponibili quando un prestito viene aperto o chiuso;
- registra e aggiorna lo stato del prestito (ad esempio: attivo, in ritardo, chiuso), gestendo anche eventuali ritardi nella restituzione.

Le principali componenti del modulo sono:

- `Prestito` e l'enumerazione `StatoPrestiti`, che descrivono il singolo prestito e il suo stato;
- `ArchivioPrestitiAttivi` e `ArchivioCronologiaPrestiti`, che distinguono i prestiti correnti da quelli conclusi e attivi;
- `PrestitiService`, che implementa tutte le regole e i controlli specifici di questa parte del dominio;
- `PrestitiViewController`, che con la sua view associata, permette al bibliotecario di visualizzare, aprire e chiudere i prestiti.

2.6.5 Modulo gestione eccezioni

Il modulo dedicato alla gestione delle eccezioni racchiude tutti gli errori specifici che possono verificarsi durante l'utilizzo dell'applicazione.

La presenza di un package separato permette di mantenere ordinato il codice dei servizi e garantire messaggi d'errore chiari e coerenti in ogni parte del sistema.

2.7 Viste architetture del sistema

Il progetto della biblioteca può essere descritto efficacemente attraverso le quattro viste principali del modello di *Kruchten*.

2.7.1 Vista logica

La vista logica rappresenta la struttura concettuale del sistema e mostra come le principali funzionalità sono raggruppate in moduli coerenti. Nel nostro progetto, il dominio è articolato attorno a tre nuclei informativi, Libri, Utenti e Prestiti, ciascuno modellato attraverso classi dedicate (entità, archivi e servizi) e supportato da un package specifico.

2.7.2 Vista di processo

La vista di processo mostra come i componenti principali interagiscono durante l'esecuzione del sistema. Il flusso tipico segue la catena:

View → Controller → Service → Archivio → Model

- La View raccoglie gli input dell'utente.
- Il Controller gestisce gli eventi e chiama i servizi.
- I Service applicano le regole di dominio (prestiti, validazioni, controlli).
- Gli Archivi gestiscono i cataloghi la persistenza tramite file.
- Le entità del Model aggiornano il loro stato.

Questa vista è ben rappresentata dai diagrammi di sequenza (presentati più avanti), che mostrano in dettaglio la collaborazione tra componenti nelle operazioni principali.

2.7.3 Vista fisica

Questa vista descrive come il software è mappato sulle risorse fisiche. Poiché l'applicazione è desktop, l'allocazione è semplice e lineare.

Il codice Java gira sulla JVM locale, JavaFX gestisce la parte grafica sulla stessa macchina e i file usati come archivi vengono salvati nel file system locale.

2.7.4 Vista di sviluppo

La vista di sviluppo descrive come il progetto è organizzato dal punto di vista degli sviluppatori, cioè come è strutturato il codice sorgente e come i componenti sono distribuiti all'interno dei package.

Nel nostro sistema, la decomposizione segue una logica object-oriented e riflette fedelmente la suddivisione per responsabilità definita in fase architetturale. Il codice è organizzato in cinque macro-package principali, ai quali si aggiunge il package *view* dedicato all'interfaccia grafica JavaFX.

Questa struttura supporta il lavoro collaborativo del team, permette una chiara distribuzione dei compiti.

2.8 L'interfaccia Archivio come decisione architetturale

L'interfaccia Archivio rappresenta una scelta architetturale strategica volta a ridurre l'accoppiamento. Essa definisce un contratto comune che tutti gli archivi devono rispettare:

- leggiDaFile(String filename)
- scriviSuFile(String filename)

Questa astrazione è presentata in Figura 2 e presenta tre vantaggi principali:

1. Uniformità: tutti gli archivi (Libri, Utenti, Prestiti Attivi, Cronologia) rispettano lo stesso schema operativo.
2. Riduzione dell'accoppiamento: i service non dipendono dall'implementazione concreta, ma solo dal comportamento previsto dall'interfaccia.
3. Estendibilità: per aggiungere un nuovo archivio basterà implementare l'interfaccia, senza modificare il resto del sistema.

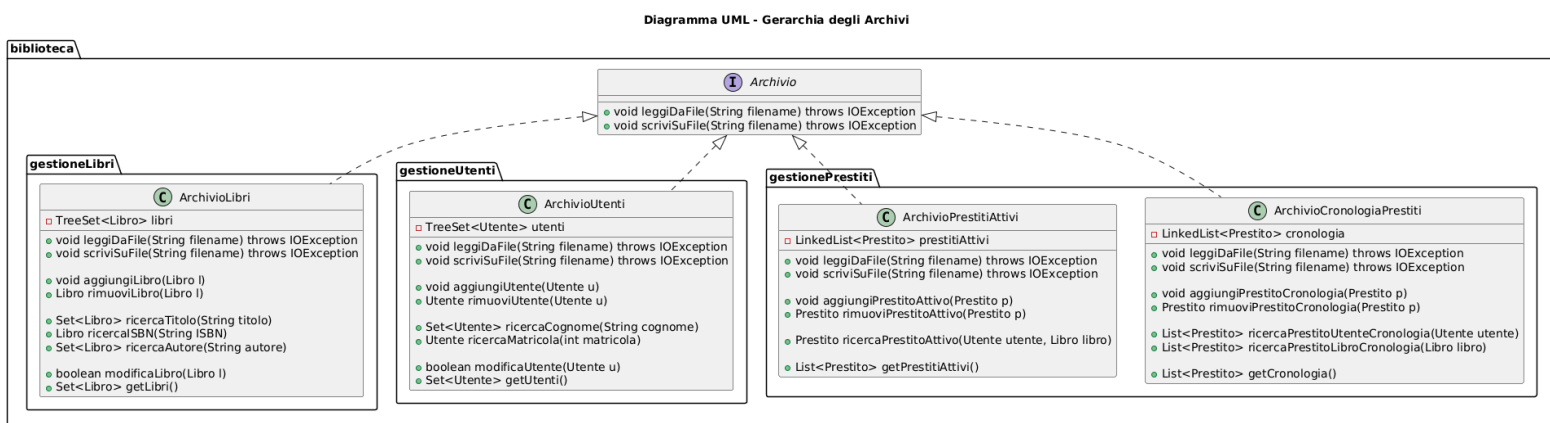


Figura 2- Diagramma UML sulla gerarchia degli archivi

2.9 Diagramma dei package dettagliato

In Figura 3 è riportato il diagramma dei package in forma dettagliata, che offre una visione complessiva dell'organizzazione del sistema e delle principali dipendenze tra i suoi moduli. In particolare, il diagramma evidenzia le relazioni che coinvolgono la gestione dei prestiti e i moduli dedicati agli utenti e ai libri, mettendo in luce una reciproca interazione coerente con la struttura del dominio applicativo.

All'interno del package principale biblioteca sono inoltre presenti la classe Main e l'interfaccia Archivio, che costituiscono elementi comuni dell'applicazione. Tali componenti non sono rappresentati attraverso i loro collegamenti, in quanto il diagramma dei package ha l'obiettivo di descrivere l'organizzazione modulare e le dipendenze tra i package, mentre le relazioni di dettaglio tra classi sono demandate ai diagrammi successivi.

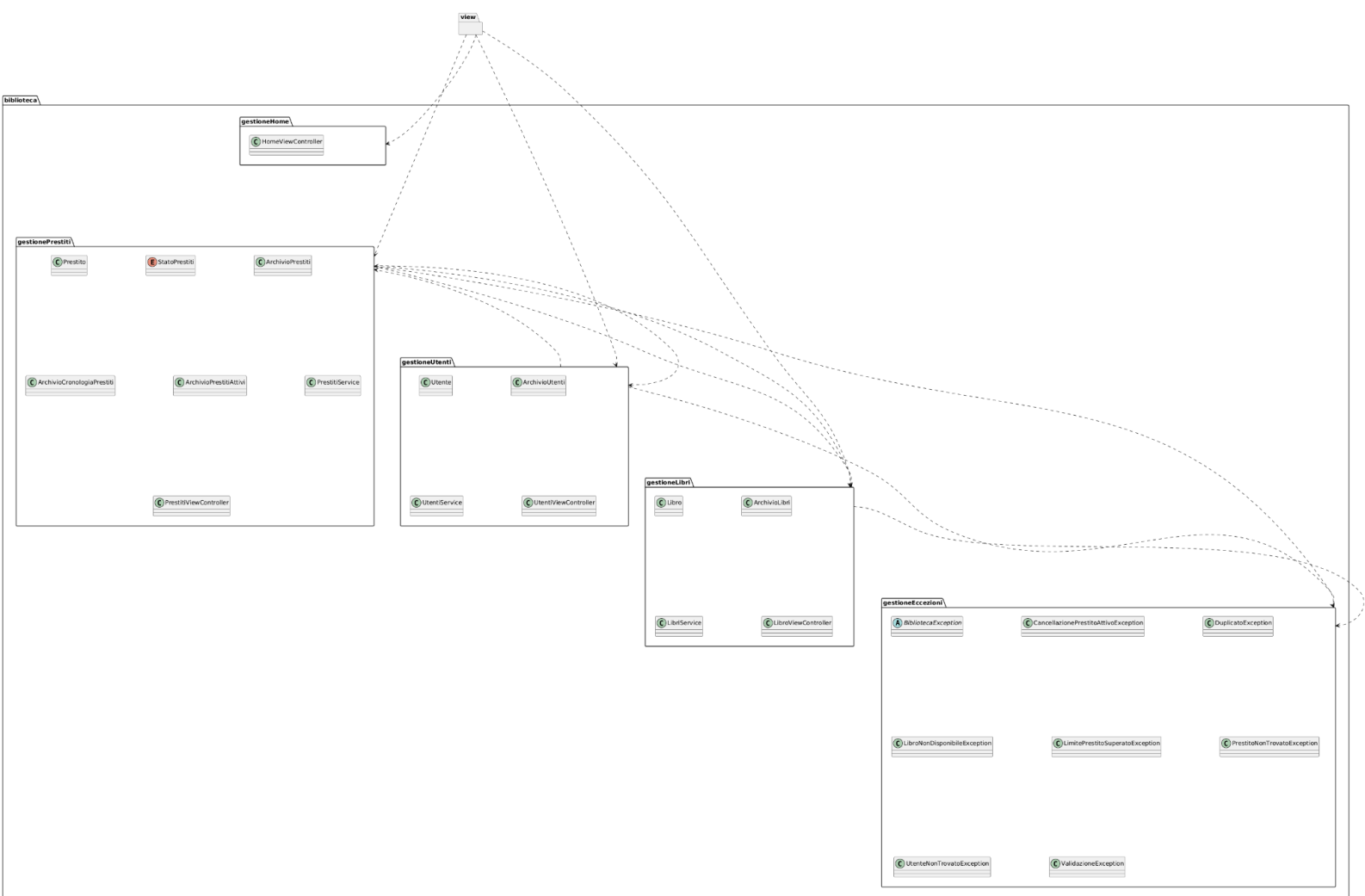


Figura 3 - Diagramma dei package

3. Progettazione di dettaglio

In questo capitolo scendiamo nel dettaglio dell'implementazione, analizzando la struttura interna delle classi principali. Per ogni componente vengono esposti il ruolo, la definizione del codice (attributi e firme dei metodi) e le motivazioni tecniche che hanno guidato le decisioni architetturali.

3.1 Modulo Gestione Libri

3.1.1 Classe Libro

Responsabilità: La classe Libro rappresenta l'unità base di tutto il catalogo. Funziona come una scheda digitale per ogni volume fisico della biblioteca. Non serve solo come archivio di dati, ma si occupa anche di gestire la quantità di copie, aggiornando il numero di volumi disponibili man mano che vengono effettuati i prestiti.

Attributi Principali:

- titolo: String
- autore: String
- annoPubblicazione: int
- isbn: String
- copieDisponibili: int

Metodi Principali:

- Costruttore: public Libro(String titolo, String autore, int annoPubblicazione, String isbn, int copieDisponibili);
- Costruttore: public Libro(String titolo, String autore, int anno, String isbn);
- metodi getter e setter per ogni attributo;
- Metodi Standard: String toString, int compareTo, boolean equals, int hashCode;

L'implementazione dell'interfaccia Comparable<Libro> tramite il compareTo definisce l'ordinamento naturale richiesto per la visualizzazione ordinata nelle tabelle e negli archivi.

3.1.2 ArchivioLibri

Responsabilità: Questa classe è il magazzino virtuale dell'applicazione. Si occupa di mantenere in memoria l'elenco di tutti i libri presenti e gestisce il salvataggio o il caricamento dei dati. Il suo compito è puramente gestionale, ovvero si assicura che i dati siano organizzati e accessibili.

Attributi Principali:

- libri: Set<Libro>;

Metodi Principali:

- Costruttore: public ArchivioLibri (String filename);
- void aggiungiLibro (Libro l);
- Libro rimuoviLibro (Libro l);
- Set<Libro> ricercaTitolo (String titolo);
- Libro ricercaISBN (String ISBN);
- Set<Libro> ricercaAutore (String autore);
- Set<Libro> getLibri ();
- boolean modificaLibro (Libro l);
- Metodi dell'interfaccia Archivio:
 - void leggiDaFile(String filename)throws IOException;
 - void scriviSuFile(String filename) throws IOException;

Per memorizzare i libri è stato scelto come metodo di ordinamento il TreeSet, perché mantiene i libri sempre in ordine alfabetico automaticamente. Inoltre impedisce che vengano inseriti duplicati semplificando così la gestione dei dati.

3.1.3 LibriService

Responsabilità: Il compito principale di questa classe è far rispettare le regole della biblioteca. Il Service decide se un'operazione è lecita o meno, bloccando inserimenti errati o cancellazioni pericolose.

Attributi Principali:

- archivioLibri: ArchivioLibri;
- archivioPrestitiAttivi: ArchivioPrestitiAttivi;

Metodi Principali:

- Costruttore: public LibriService(ArchivioLibri archivioLibri, ArchivioPrestitiAttivi archivioPrestitiAttivi);
- public void registraLibro(Libro l) throws ValidazioneException, DuplicatoException;
- public Libro eliminaLibro(Libro l) throws CancellazionePrestitoAttivoException;
- public void modificaLibro(Libro l, String isbnOriginale) throws ValidazioneException, DuplicatoException;
- public Set<Libro> ricercaLibro(String titolo, String autore, String isbn);
- public Set<Libro> visualizzaLibri();

3.1.4 Libro ViewController

Responsabilità: Questa classe funge da controllore per l'interfaccia di gestione dei libri. Il suo compito principale è duplice: da un lato gestisce le operazioni sul catalogo (aggiunta, rimozione, ricerca) interpretando gli input dell'utente; dall'altro, gestisce la navigazione

dell'intera applicazione, permettendo il passaggio tra le diverse schermate all'interno della stessa finestra.

Attributi Grafici:

- **Campi Input:**
 - txtTitolo: TextField;
 - txtAutori: TextField;
 - txtISBN: TextField;
 - txtCopie: TextField;
 - txtAnno: TextField
- **Bottoni:**
 - addButton: Button;
 - removeButton: Button;
 - searchButton: Button;
 - viewButton: Button;
- **Tabelle:**
 - libroTable: TableView<Libro>;
 - colTitolo: TableColumn<Libro, String>;
 - colAutore: TableColumn<Libro, String>;
 - colISBN: TableColumn<Libro, String>;
 - colNumCopie: TableColumn<Libro, Integer> ;
 - colAnno: TableColumn<Libro, Integer> ;
- **Elementi di Navigazione:**
 - homeID: Label;
 - utentiID: Label;
 - prestitiID: Label;
- **Logica:**
 - private LibriService libroService;
 - private ObservableList<Libro> listaLibri;
 - private String isbnOriginale;

Metodi Principali:

- public void initialize(URL url, ResourceBundle rb);
- **Navigazione Menu:**
 - void clickUtenti (MouseEvent event);
 - void clickPrestiti(MouseEvent event);
 - void clickHome(MouseEvent event);
- **Metodo di utilità:**
 - Stage getStage(Label label);
- **Gestione Operazioni:**
 - void onAggiungiLibro(ActionEvent event);
 - void onRimuoviLibro(ActionEvent event);
 - void onRicercaLibro(ActionEvent event);
 - void onVisualizzaLibri(ActionEvent event);
- **Gestione delle modifiche in tabella:**
 - private void onModificaTitolo(TableColumn.CellEditEvent<Libro, String> event);

- private void onModificaAutore(TableColumn.CellEditEvent<Libro, String> event);
- private void onModificaISBN(TableColumn.CellEditEvent<Libro, String> event);
- private void onOriginaleISBN(TableColumn.CellEditEvent<Libro, String> event);
- private void onModificaCopie(TableColumn.CellEditEvent<Libro, Integer> event);
- private void onModificaAnno(TableColumn.CellEditEvent<Libro, Integer> event);
- **Gestione Alert:**
 - private boolean alertConfermaEliminazione();
 - private void alertErrore(String messaggio);
 - private void alertConferma(String messaggio);

La navigazione non apre nuove finestre, ma sostituisce il contenuto della finestra esistente (Stage). Per evitare di duplicare il codice necessario a recuperare la finestra corrente, è stato implementato il metodo di supporto `getStage`, che estrae lo Stage a partire dall'etichetta cliccata. Successivamente, viene usato `FXMLLoader` per caricare il file `.xml` della nuova vista.

3.2 Modulo Gestione Utenti

3.2.1 Classe Utente

Responsabilità: Questa classe è la rappresentazione digitale dello studente/professore iscritto alla biblioteca. Utente funge da fascicolo personale, ogni oggetto utente memorizza quali libri ha in prestito in quel momento. Ciò permette al sistema di controllare se una persona può prendere altri libri in prestito o se ha raggiunto il limite.

Attributi Principali:

- nome: String;
- cognome: String;
- matricola: String;
- email: String;
- prestitiAttivi: List<Prestito>;

Metodi Principali:

- Costruttore: public Utente(String nome, String cognome, String matricola, String email);
- Metodi getter e setter per ogni attributo;
- void aggiungiPrestitoAttivo (Prestito p);
- void rimuoviPrestitoAttivo (Prestito p);
- public List<Prestito> getPrestitiAttivi ();

- Metodi Standard: String toString, boolean equals, int hashCode(), int compareTo;
- public String getPrestitiTitolo();
- public String getPrestitiDataRest();

Grazie a compareTo, gli utenti vengono ordinati automaticamente per Cognome(e poi eventualmente per nome) nelle liste.

3.2.2 ArchivioUtenti

Responsabilità: Questa classe funge da registro degli iscritti. Si occupa di mantenere la lista degli utenti in memoria e di gestire il salvataggio fisico su file. Come per i libri, il suo ruolo è puramente "meccanico": esegue gli ordini di inserimento o cancellazione, lasciando ai Service il compito di verificare se tali ordini sono leciti.

Attributi Principali:

- utenti: Set<Utente>;

Metodi Principali:

- Costruttore: public ArchivioUtenti (String filename);
- void aggiungiUtente (Utente u);
- Libro rimuoviUtente (Utente u);
- Set<Utente> ricercaCognome (String cognome);
- Utente ricercaMatricola (String matricola);
- Set<Libro> getUtenti ();
- boolean modificaUtente (Utente u);
- Metodi dell'interfaccia Archivio:
 - void leggiDaFile(String filename) throws IOException;
 - void scriviSuFile(String filename) throws IOException;

Anche qui è stato scelto il TreeSet per mantenere l'elenco degli iscritti sempre ordinato alfabeticamente e per impedire duplicati a livello di struttura dati.

3.2.3 UtentiService

Responsabilità: Verifica la correttezza delle operazioni sugli utenti. Si assicura che i dati inseriti siano sensati e protegge la coerenza del sistema impedendo azioni rischiose, come cancellare un utente che ha ancora dei libri della biblioteca a casa.

Attributi Principali:

- archivioUtenti: ArchivioUtenti;
- filename = String;

Metodi Principali:

- Costruttore: `public UtentiService(ArchivioUtenti archivioUtenti);`
- `void registraUtente(Utente u) throws ValidazioneException, DuplicatoException;`
- `Utente eliminaUtente(Utente u) throws CancellazionePrestitoAttivoException;`
- `public Set<Utente> ricercaUtente(String cognome, String matricola) throws ValidazioneException, UtenteNonTrovatoException;`
- `public void modificaUtente(Utente utente, String matricolaOriginale) throws ValidazioneException, DuplicatoException;`
- `Set<Utente> visualizzaUtenti ();`

3.2.4 UtentiViewController

Responsabilità: Questa classe è il pannello di controllo per la gestione degli iscritti. Svolge due funzioni primarie: gestisce l'interfaccia operativa per l'anagrafica (inserimento, cancellazione e ricerca utenti) e, parallelamente, garantisce la navigazione verso le altre sezioni del gestionale all'interno della stessa finestra.

Attributi Grafici:

- **Campi Input:**
 - `txtNome: TextField;`
 - `txtCognomei: TextField;`
 - `txtMatricola: TextField;`
 - `txtEmail: TextField;`
- **Bottoni:**
 - `addButton: Button;`
 - `removeButton: Button;`
 - `searchButton: Button;`
 - `viewButton: Button;`
- **Tabelle:**
 - `utenteTable: TableView<Utente>;`
 - `colNome: TableColumn<Utente, String>;`
 - `colCognome: TableColumn<Utente, String>;`
 - `colMatricola: TableColumn<Utente, Integer>;`
 - `colEmail: TableColumn<Utente, String>;`
 - `colPrestit: TableColumn<Utente, String>;`
 - `colDataRest: TableColumn<Utente, LocalDate>;`
- **Elementi di Navigazione:**
 - `homeID: Label;`
 - `libriID: Label;`
 - `prestitID: Label;`
- **Logica:**
 - `private UtentiService utenteService;`
 - `private ObservableList<Utente> listaUtenti;`
 - `private String matricolaOriginale;`

Metodi Principali:

- `public void initialize(URL url, ResourceBundle rb);`

- **Navigazione Menu:**
 - void clickLibri(MouseEvent event);
 - void clickPrestiti(MouseEvent event);
 - void clickHome(MouseEvent event);
- **Metodo di utilità:**
 - Stage getStage(Label label);
- **Gestione Operazioni:**
 - void onAggiungiUtente(ActionEvent event);
 - void onRimuoviUtente(ActionEvent event);
 - void onRicercaUtente(ActionEvent event);
 - void onVisualizzaUtenti(ActionEvent event);
- **Metodo di utilità:**
 - Stage getStage(Label label);
- **Gestione delle modifiche in tabella:**
 - private void onModificaNome(TableColumn.CellEditEvent<Utente, String> event);
 - private void onModificaCognome(TableColumn.CellEditEvent<Utente, String> event);
 - private void onModificaEmail(TableColumn.CellEditEvent<Utente, String> event);
 - private void onModificaMatricola(TableColumn.CellEditEvent<Utente, String> event);
 - private void onOriginaleMatricola(TableColumn.CellEditEvent<Utente, String> event);
- **Gestione Alert:**
 - private boolean alertConfermaEliminazione();
 - private void alertErrore(String messaggio);
 - private void alertConferma(String messaggio);

La navigazione non apre nuove finestre, ma sostituisce il contenuto della finestra esistente (Stage). Per evitare di duplicare il codice necessario a recuperare la finestra corrente, è stato implementato il metodo di supporto getStage, che estrae lo Stage a partire dall'etichetta cliccata. Successivamente, viene usato FXMLLoader per caricare il file .fxml della nuova vista.

3.3 Modulo Gestione Prestiti

3.3.1 Classe Prestito

Responsabilità: Questa classe rappresenta l'operazione di prestito vera e propria. Ovvero tiene traccia del tempo (quando è iniziato, quando deve finire) e dello stato attuale (se è tutto ok o se c'è un ritardo) del prestito.

Attributi Principali:

- id: final int;
- utente: Utente;

- libro: Libro;
- dataInizio: LocalDate;
- dataFine: LocalDate;
- stato: StatoPrestiti;

Metodi Principali:

- public Prestito(int id, Utente utente, Libro libro, LocalDate dataInizio, LocalDate dataFine, StatoPrestiti stato);
- Metodi getter e setter per ogni attributo;

3.3.2 StatoPrestiti

Responsabilità: Questa classe enum è una semplice lista di etichette che definisce le uniche tre situazioni possibili per un prestito.

Valori:

- ATTIVO: Il libro è fuori in prestito e non è ancora scaduto.
- RITARDO: La data di scadenza è passata e il libro non è tornato.
- CHIUSO: Il libro è stato restituito.

3.3.3 ArchivioPrestitiAttivi

Responsabilità: Questa classe memorizza esclusivamente i prestiti che sono attualmente in corso. Il suo scopo principale è fornire risposte immediate quando bisogna verificare se un utente può prendere un altro libro: controllando la lunghezza della lista in questo archivio, il sistema sa subito se l'utente ha raggiunto il limite massimo.

Attributi Principali:

- prestitiAttivi: List<PrestitiAttivi>;

Metodi Principali:

- Costruttore: public ArchivioPrestitiAttivi (String filename, ArchivioUtenti archivioUtenti, ArchivioLibri archivioLibri);
- void aggiungiPrestitoAttivo (Prestito p);
- Prestito rimuoviPrestitoAttivo (Prestito p);
- Prestito ricercaPrestitoAttivo (Utente utente, Libro libro);
- List<Prestito> getPrestitiAttivi ();
- Metodi dell'interfaccia Archivio:
 - void leggiDaFile(String filename) throws IOException;
 - void scriviSuFile(String filename) throws IOException;

Abbiamo scelto una lista concatenata perché è molto efficiente nelle operazioni di aggiunta e rimozione, che sono frequenti in questo archivio.

3.3.4 ArchivioCronologiaPrestiti

Responsabilità: Questa classe funge da memoria storica dell'applicazione. Conserva la traccia di tutte le operazioni effettuate, sia quelle ancora aperte che quelle concluse. A differenza dell'archivio attivo, che è dinamico, questo tende a crescere nel tempo e serve principalmente per consultazioni, statistiche o per ricostruire la storia di un libro o di un utente.

Attributi Principali:

- cronologia: List<Prestiti>;

Metodi Principali:

- Costruttore: public ArchivioCronologiaPrestiti (String filename, ArchivioUtenti archivioUtenti, ArchivioLibri archivioLibri);
- void aggiungiPrestitoCronologia (Prestito p);
- Prestito rimuoviPrestitoCronologia (Prestito p);
- List<Prestito> ricercaPrestitoUtenteCronologia (Utente utente);
- List<Prestito> ricercaPrestitoLibroCronologia (Libro libro);
- Prestito ricercaPrestitoUtenteLibro (Utente utente, Libro libro);
- List<Prestito> getCronologia();
- Metodi dell'interfaccia Archivio:
 - void leggiDaFile(String filename) throws IOException;
 - void scriviSuFile(String filename) throws IOException

Anche qui si usa una LinkedList perché la priorità è mantenere l'ordine cronologico di inserimento. I nuovi prestiti vengono semplicemente aggiunti in coda alla lista.

3.3.5 PrestitiService

Responsabilità: È l'unica classe che ha accesso a tutti e quattro gli archivi contemporaneamente (Libri, Utenti, Prestiti Attivi, Cronologia). Il suo compito è coordinare le operazioni complesse che coinvolgono più entità.

Attributi Principali:

- archivioLibri: ArchivioLibri;
- archivioUtenti: ArchivioUtenti;
- archivioPrestitiAttivi: ArchivioPrestitiAttivi;
- archivioCronologia: ArchivioCronologiaPrestiti;

Metodi Principali:

- Costruttore: public PrestitiService(ArchivioLibri archivioLibri, ArchivioUtenti archivioUtenti, ArchivioPrestitiAttivi archivioPrestitiAttivi, ArchivioCronologiaPrestiti archivioCronologia)

- void registraPrestitoAttivo(Prestito p) throws ValidazioneException, LibroNonDisponibileException, LimitePrestitoSuperatoException;
- Prestito eliminaPrestitoAttivo(Prestito p) throws ValidazioneException;
- Prestito cercaPrestitoAttivo(Utente utente, Libro libro) throws ValidazioneException, PrestitoNonTrovatoException;
- List<Prestito> visualizzaPrestitiAttivi ();
- void registraPrestitoCronologia(Prestito p) throws ValidazioneException;
- Prestito eliminaPrestitoCronologia(Prestito p) throws ValidazioneException;
- List<Prestito> cercaPrestitoUtenteCronologiaPerUtente(Utente utente) throws ValidazioneException, PrestitoNonTrovatoException;
- List<Prestito> ricercaPrestitoLibroCronologiaPerLibro(Libro libro) throws ValidazioneException, PrestitoNonTrovatoException;
- List<Prestito> visualizzaCronologia ();

3.3.6 PrestitiViewController

Responsabilità: Questa classe rappresenta il pannello di controllo per la gestione dei movimenti dei libri. È l'interfaccia più ricca del sistema perché permette al bibliotecario di svolgere tre compiti distinti: registrare l'uscita di un libro (nuovo prestito), registrare il rientro (restituzione) e consultare sia la situazione attuale che lo storico completo. Come gli altri controller, gestisce anche la navigazione verso le altre sezioni del gestionale.

Attributi Grafici:

- **Campi Input:**
 - textISBN: TextField;
 - txtMatricola1: TextField;
 - txtDataRestituzione: TextField;
- **Bottoni:**
 - addButton: Button;
 - cronButton: Button;
 - searchButton: Button;
 - viewButton: Button;
 - backButton: Button
- **Tabelle:**
 - prestitoTable: TableView<Prestito>;
 - colID: TableColumn<Prestito, Integer>;
 - colNome: TableColumn<Prestito, String>;
 - colCognome: TableColumn<Prestito, String>;
 - colMatricola: TableColumn<Prestito, String>;
 - colTitolo: TableColumn<Prestito, String>;
 - colAutori: TableColumn<Prestito, String>;
 - colEmail: TableColumn<Prestito, String>;
 - colISBN: TableColumn<Prestito, String>;
 - colDataInizio: TableColumn<Prestito, LocalDate>;
 - colDataFine: TableColumn<Prestito, LocalDate>;
 - colStato: TableColumn<Prestito, StatoPrestiti>;

- **Elementi di Navigazione:**

- homeID: Label;
- utentiID: Label;
- libriID: Label;

Metodi Principali:

- `public void initialize(URL url, ResourceBundle rb);`
- **Metodo di utilità:**
 - `Stage getStage(Label label);`
- **Navigazione Menu:**
 - `void clickLibri(MouseEvent event);`
 - `void clickUtenti(MouseEvent event);`
 - `void ClickHome(MouseEvent event);`
- **Gestione Operazioni:**
 - `void onAggiungiPrestito(ActionEvent event);`
 - `void onRicercaPrestito(ActionEvent event);`
 - `void onRestituzioneLibro(ActionEvent event);`
 - `void onVisualizzaPrestito(ActionEvent event);`
 - `void onVisualizzaCronologia(ActionEvent event);`
- **Gestione Alert:**
 - `private void alertInfo (String msg);`
 - `private void alertErrore (String msg);`

La navigazione non apre nuove finestre, ma sostituisce il contenuto della finestra esistente (Stage). Per evitare di duplicare il codice necessario a recuperare la finestra corrente, è stato implementato il metodo di supporto `getStage`, che estrae lo Stage a partire dall'etichetta cliccata. Successivamente, viene usato `FXMLLoader` per caricare il file `.fxml` della nuova vista.

Un'attenzione particolare è stata rivolta anche alla persistenza dei dati e alla gestione degli identificativi. Sono stati quindi aggiunti specifici metodi che permettono di ripristinare correttamente gli ID dei prestiti all'avvio dell'applicazione, sulla base delle informazioni salvate su file. Questa scelta garantisce la continuità dei dati anche dopo la chiusura del programma, evitando duplicazioni.

Inoltre sono stati introdotti ulteriori controlli di verifica, fondamentali per il corretto funzionamento del sistema. In particolare, sono stati implementati metodi che consentono di determinare il numero di prestiti associati a un utente, di stabilire se un prestito risulta scaduto attraverso il confronto tra le date previste e la data corrente, e di verificare se un prestito è ancora attivo. Quest'ultima informazione risulta essenziale per impedire operazioni non ammesse, come l'eliminazione di un utente o di un libro coinvolto in un prestito non ancora concluso.

3.4 Modulo GestioneHome

3.4.1 HomeViewController

Responsabilità: È il controller responsabile della dashboard principale (Home) e il suo compito esclusivo è gestire la navigazione. Non elabora dati di business, ma funge da snodo centrale per indirizzare l'utente verso le tre aree operative del sistema (Libri, Utenti, Prestiti), garantendo un passaggio fluido tra le diverse schermate.

Attributi Principali:

- LibriID: Label;
- UtentiID: Label;
- PrestitiID: Label;

Metodi Principali:

- public void initialize(URL url, ResourceBundle rb);
- **Metodo di utilità:**
 - Stage getStage(Label label);
- **Gestore degli eventi di navigazione:**
 - void clickLibri(MouseEvent event);
 - void clickUtenti(MouseEvent event);
 - void clickPrestiti(MouseEvent event);

Abbiamo scelto di dedicare un controller specifico solo alla Home e alla navigazione. In questo modo, la logica di "spostamento" tra le schermate è isolata dalla logica di "gestione" dei dati. Se in futuro volessimo cambiare il modo in cui ci si sposta nell'app, basterà modificare solo questa classe.

@FXML: Queste etichette servono a collegare il codice Java direttamente agli elementi grafici disegnati con SceneBuilder. Grazie a questo meccanismo, quando l'utente clicca su una voce del menu a video, il programma sa esattamente quale metodo Java eseguire.

3.5 Modulo GestioneEccezioni

Responsabilità: Questo modulo serve a gestire gli imprevisti. Il suo compito è fornire al sistema un vocabolario preciso per descrivere cosa va storto. Invece di usare errori generici di Java, abbiamo creato una famiglia di errori "personalizzati" che descrivono esattamente i problemi del nostro dominio (es. "Libro non disponibile" o "Matricola duplicata").

- **DuplicatoException:** Segnala il tentativo di inserire una chiave già esistente (es. due libri con lo stesso ISBN).
- **ValidazioneException:** Viene lanciata quando l'utente lascia campi vuoti o inserisce formati sbagliati (es. lettere al posto di numeri nell'anno).
- **LibroNonDisponibileException:** impedisce il prestito se le copie disponibili sono zero.
- **LibroNonTrovatoException, UtenteNonTrovatoException, PrestitoNonTrovatoException:** Segnala il tentativo di inserire una chiave già esistente (es. due libri con lo stesso ISBN).
- **LimitePrestitoSuperatoException:** Blocca l'operazione se l'utente ha già 3 libri in carico.

- **CancellazionePrestitoAttivoException:** Blocca la rimozione di un utente o di un libro se sono coinvolti in un prestito non ancora chiuso.

Tutte le eccezioni sopra citate ereditano dalla classe padre comune `BibliotecaException`. Ciò risulta utile qualora volessimo catturare tutti gli errori della biblioteca in un colpo solo, basterebbe scrivere `catch (BibliotecaException e)`, senza dover elencare tutte le sottoclassi una per una.

3.6 Applicazione dei principi SOLID

In questa sezione analizziamo come l'architettura del sistema sia stata modellata seguendo i principi di buona programmazione (SOLID) per garantire che il software sia facile da mantenere ed estendere nel tempo.

S — Single Responsibility Principle

Ogni classe deve avere un solo motivo per cambiare, ovvero una sola responsabilità ben definita.

- Le classi `Service` si occupano solo della logica e delle regole.
- Le classi `Archivio` si occupano solo di memorizzare i dati.
- Le classi `ViewController` si occupano solo di gestire la grafica e l'input dell'utente. In questo modo, se un domani dovessimo cambiare l'aspetto grafico, modificheremo solo il `ViewController` senza rischiare di rompere la logica di salvataggio.

O — Open/Closed Principle

Il sistema è progettato per crescere senza dover riscrivere il codice esistente. Ad esempio, la gestione delle eccezioni (`BibliotecaException`) è strutturata in una gerarchia. Se in futuro volessimo aggiungere un nuovo tipo di errore (es. "Utente moroso"), basterebbe creare una nuova classe che estende quella base, senza dover modificare il meccanismo generale di cattura degli errori.

L — Liskov Substitution Principle

Le sottoclassi devono poter sostituire la classe base senza alterare il funzionamento del programma. Questo principio è visibile nell'uso dell'interfaccia `Archivio`. Qualsiasi classe che implementa `Archivio` garantisce di avere i metodi `leggiDaFile` e `scriviSuFile` funzionanti. Il sistema si aspetta un comportamento standard da tutti gli archivi e le implementazioni concrete rispettano questa aspettativa.

I — Interface Segregation

Per questo lavoro si utilizza un'interfaccia semplice, `Archivio`, evitando di aggiungere metodi inutili che le classi non devono implementare. Ad esempio, `Libro` e `Utente` implementano l'interfaccia `Comparable` solo per poter essere ordinate; in questo modo non sono costrette a gestire operazioni di persistenza o di validazione che non rientrano nelle loro responsabilità.

D — Dependency Inversion

Le classi di alto livello non dovrebbero dipendere dai dettagli di basso livello, ma entrambe dovrebbero fare riferimento ad astrazioni. Questo è il principio che guida l'uso dell'interfaccia Archivio. I service, infatti, non dipendono da come i dati vengono salvati concretamente, ma interagiscono solo con l'astrazione dell'archivio.

3.7 Qualità del software e contenimento del debito tecnico

In questa sezione finale spieghiamo come abbiamo lavorato per evitare che il codice diventi "ingestibile" in futuro. L'obiettivo è stato mantenere basso il debito tecnico, ovvero la complessità inutile che rende difficili le modifiche successive.

Strategie adottate: Per garantire che il software rimanga pulito e facile da aggiornare, abbiamo seguito poche ma rigide regole:

- Ordine prima di tutto: Ogni classe ha il suo posto preciso. Non abbiamo mischiato la grafica con la logica, né la gestione dei file con le regole dei prestiti. Questo rende il codice leggibile.
- Isolamento delle modifiche: Grazie all'architettura che abbiamo scelto, se dobbiamo cambiare qualcosa, sappiamo esattamente dove mettere le mani senza rischiare di rompere altre parti del programma.

Questo modo di progettare richiede un po' più di attenzione all'inizio, ma riduce drasticamente il rischio di introdurre bug quando si fanno aggiornamenti e rende molto più semplice testare il funzionamento delle singole parti.



4. Modello statico

Il modello statico descrive la struttura interna del sistema, mettendo in evidenza gli elementi fondamentali del dominio e le relazioni tra essi. La progettazione è stata suddivisa in due livelli:

1. un diagramma essenziale, che raccoglie esclusivamente gli oggetti del dominio e gli archivi che li gestiscono;
2. un diagramma completo, che estende il primo introducendo i servizi applicativi, le eccezioni personalizzate e il layer di interfaccia grafica.

Questa scelta consente di comprendere prima la logica fondamentale descritta nello SRS (prestiti, utenti, libri, vincoli e regole) per poi contestualizzarla all'interno dell'architettura software effettiva.

4.1 Diagramma delle classi – modello essenziale

Il primo diagramma descrive la parte più fondamentale del sistema, cioè l'insieme delle classi che rappresentano gli elementi centrali del dominio applicativo e gli archivi che ne consentono la gestione.

Questa sezione costituisce la traduzione più diretta dei requisiti individuati nello SRS, perché raccoglie tutti gli aspetti indispensabili al funzionamento della biblioteca digitale.

Dunque il diagramma in Figura 4 costituisce la base concettuale dell'intero sistema: è qui che vengono definite le regole e i rapporti fondamentali tra utenti, libri e prestiti, prima di introdurre ulteriori elementi architetturali che, nel diagramma successivo, si occuperanno della logica applicativa e dell'interazione con l'interfaccia grafica.

Le classi fondamentali individuate sono:

Classi di dominio

- Libro: rappresenta un volume presente in biblioteca; contiene titolo, autore, ISBN, anno di pubblicazione e numero di copie disponibili.
- Utente: modella la persona registrata presso il sistema; include nome, cognome, matricola e indirizzo e-mail istituzionale, oltre alla lista dei prestiti attivi.
- Prestito: rappresenta l'atto del prestito di un libro, includendo data di inizio, data prevista di restituzione, eventuale data effettiva e stato del prestito.
- StatoPrestiti: enumerazione che definisce i possibili stati (ATTIVO, RITARDO, CHIUSO).

Archivi

Gli archivi permettono di organizzare e mantenere le istanze delle classi di dominio:

- ArchivioLibri, responsabile della gestione, ricerca e memorizzazione dei libri;
- ArchivioUtenti, che mantiene l'insieme degli utenti registrati;
- ArchivioPrestitiAttivi, che conserva i soli prestiti correnti;
- ArchivioCronologiaPrestiti, dove vengono inseriti i prestiti chiusi.

Tutti gli archivi implementano l'interfaccia Archivio, la quale presenta esclusivamente metodi di scrittura e salvataggio dei dati.

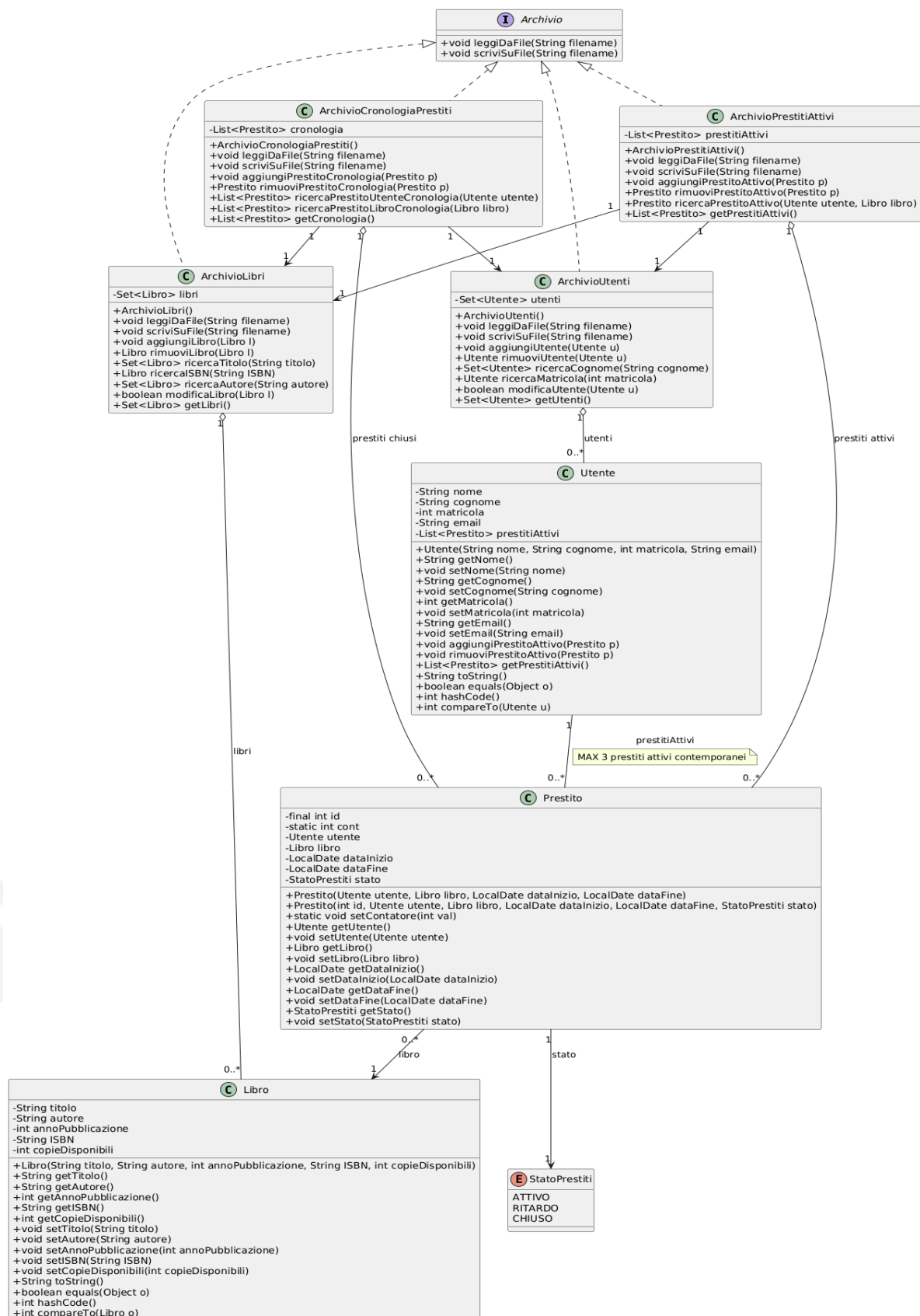


Figura 4 - Diagramma delle classi essenziale

4.2 Diagramma delle classi – modello completo

La Figura 5 presenta il diagramma delle classi completo del sistema, che estende il modello di dominio introducendo gli elementi necessari alla realizzazione concreta dell'applicazione in Java e JavaFX. Accanto alle classi fondamentali del dominio (Libro, Utente, Prestito e StatoPrestiti), il diagramma include infatti i livelli applicativi e infrastrutturali che consentono di separare in modo chiaro responsabilità e ruoli.

In particolare, sono rappresentate le classi Service (LibriService, UtentiService e PrestitiService), che costituiscono il cuore della logica applicativa. Queste classi si occupano della validazione dei dati, dell'applicazione delle regole di business (come il limite massimo di prestiti attivi o la disponibilità delle copie) e del coordinamento tra gli archivi, evitando che tali responsabilità ricadano sulle classi di dominio o sull'interfaccia grafica.

Il livello di presentazione è modellato tramite i ViewController JavaFX, responsabili esclusivamente della gestione dell'interazione con l'utente e della delega delle operazioni ai rispettivi service. Questa scelta riflette un'architettura di tipo MVC, in cui l'interfaccia rimane disaccoppiata dalla logica applicativa e dai dati.

Il diagramma evidenzia inoltre il ruolo centrale degli archivi (ArchivioLibri, ArchivioUtenti, ArchivioPrestitiAttivi e ArchivioCronologiaPrestiti), che fungono da contenitori dei dati e implementano l'interfaccia Archivio. Quest'ultima definisce le operazioni comuni di lettura e scrittura su file, consentendo una gestione uniforme della persistenza senza esporre i dettagli di implementazione agli altri livelli del sistema.

Completa il modello la gerarchia delle eccezioni, organizzata a partire dalla classe astratta BibliotecaException. Le eccezioni specifiche permettono di gestire in modo controllato le principali condizioni di errore del dominio (duplicati, dati non validi, prestiti non trovati, superamento dei limiti, indisponibilità dei libri), migliorando la robustezza del sistema e la chiarezza della comunicazione degli errori.



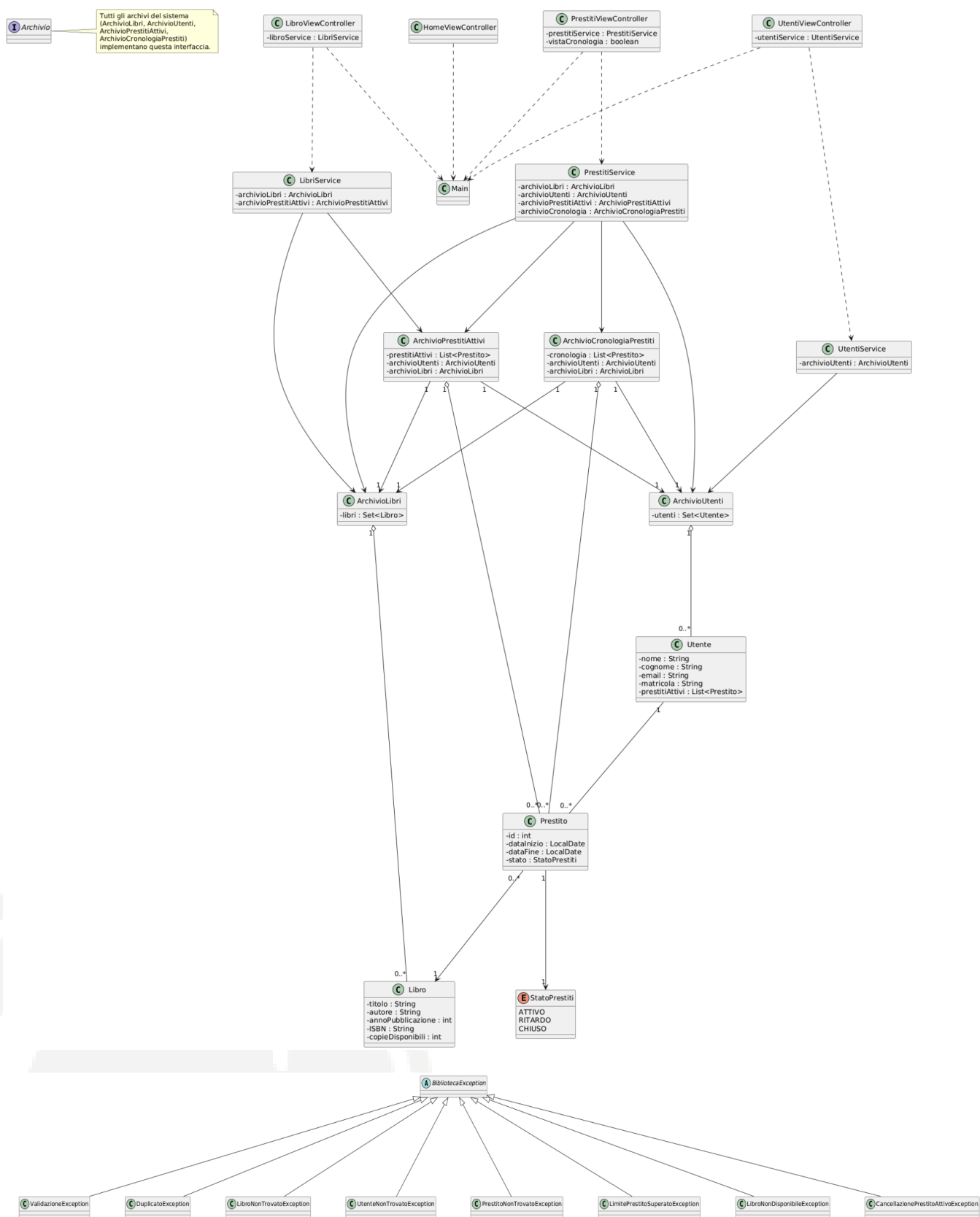


Figura 5- Diagramma delle classi completo

4.3 Descrizione dettagliata delle relazioni

4.3.1 Associazioni

Utente – Prestito (1 – 0..*)

La relazione tra *Utente* e *Prestito* è di tipo uno-a-molti, poiché ogni utente può avere più prestiti attivi contemporaneamente (nello specifico massimo 3 prestiti attivi contemporaneamente). La classe *Utente* mantiene infatti una lista di prestiti, che inizialmente è vuota al momento della registrazione. D'altra parte, ogni prestito è sempre riferito a un solo utente, in quanto rappresenta un'operazione identificata univocamente. Questa relazione è l'unica bidirezionale del modello, ed è strutturata così per ragioni funzionali:

- l'utente deve poter accedere rapidamente ai propri prestiti attivi;
- ogni prestito deve indicare con precisione l'utente a cui è stato assegnato.

Prestito – Libro (0..* – 1)

La relazione tra *Prestito* e *Libro* è una classica molti-a-uno: nel tempo un libro può essere coinvolto in numerosi prestiti, mentre ogni prestito fa riferimento a un unico volume.

La responsabilità di verificare se il libro è disponibile, e quindi se è possibile emettere un nuovo prestito, appartiene ai *Service*, che gestiscono l'aggiornamento del numero di copie disponibili e impediscono la creazione di prestiti non validi.

Prestito – StatoPrestiti (1 – 1)

Ogni prestito è associato a uno stato, rappresentato dall'enumerazione *StatoPrestiti*.

Lo stato del prestito svolge un ruolo centrale nella logica applicativa: determina se l'operazione è ancora attiva, se deve essere segnalata in ritardo oppure se può essere archiviata nella cronologia.

La relazione è unidirezionale poiché lo stato non necessita di conoscere il prestito a cui è associato.

Archivi – Archivi

Gli archivi dei prestiti (*ArchivioPrestitiAttivi* e *ArchivioCronologiaPrestiti*) dipendono dagli archivi di libri e utenti per mantenere la coerenza dei riferimenti ai dati di dominio. Essi utilizzano tali archivi per accedere alle informazioni sugli utenti e sui libri associati ai prestiti, senza gestirne direttamente il ciclo di vita.

Service – Archivi

Le classi *service* dipendono dagli archivi per eseguire le operazioni sui dati persistenti (ricerche, salvataggi, aggiornamenti).

Si tratta di una relazione di dipendenza: i *service* non creano né gestiscono il ciclo di vita degli archivi, ma li utilizzano come strumenti per accedere alle informazioni.

4.3.2 Dipendenze

Controller – Service

I controller dipendono dai service per eseguire le operazioni di business richieste dall'interfaccia grafica. Essi non accedono direttamente agli archivi, ma delegano ai service la gestione della logica applicativa.

Controller – Main

I controller dipendono dalla classe *Main* per l'accesso ai componenti condivisi dell'applicazione e per la gestione dell'inizializzazione delle viste. La classe *Main* fornisce il contesto applicativo e, a sua volta, inizializza gli archivi necessari all'avvio del sistema. Sono pertanto presenti ulteriori dipendenze tra *Main* e gli archivi, che non sono state rappresentate nel diagramma al fine di migliorarne la leggibilità.

4.3.3 Aggregazioni

Libro – ArchivioLibri

L'archivio conserva l'insieme dei libri registrati, ma i libri vengono creati in modo indipendente tramite la loro classe. L'archivio si limita a organizzarli e a permetterne la consultazione.

Utente – ArchivioUtenti

Gli utenti non dipendono dall'archivio per la loro esistenza: quest'ultimo li raccoglie e li rende accessibili, ma non ne controlla il ciclo di vita. Per questo motivo la relazione è modellata come aggregazione.

Prestito – ArchivioPrestitiAttivi

Anche i prestiti attivi sono entità autonome: vengono creati all'interno della logica di business e successivamente inseriti nell'archivio. L'archivio funge da contenitore dinamico, che riflette la situazione corrente dei prestiti attivi.

Prestito – ArchivioCronologiaPrestiti

L'archivio della cronologia gestisce i prestiti già conclusi, ma non ne determina la creazione né la distruzione. I prestiti, dopo essere stati chiusi, vengono semplicemente trasferiti nell'archivio storico, che ha il ruolo di conservarli e renderli consultabili.

4.4 Analisi della coesione e dell'accoppiamento

Questa sezione analizza la qualità strutturale del modello secondo i principi dell'ingegneria del software, in particolare:

- Tipologia di coesione (quanto una classe è focalizzata su un singolo compito)
- Tipologia di accoppiamento (quanto una classe dipende da altre)

4.4.1 Coesione

Nel sistema è stata ricercata una coesione alta in tutte le componenti principali, condizione essenziale per ottenere classi semplici da comprendere e modificare.

Classi di dominio (Libro, Utente, Prestito): coesione funzionale

Le classi di dominio presentano coesione funzionale, la forma più forte, perché ogni classe realizza un singolo compito ben definito:

- *Libro* gestisce esclusivamente i dati relativi a un volume.
- *Utente* conserva le informazioni anagrafiche e la lista dei prestiti attivi.
- *Prestito* rappresenta un'operazione di prestito con le sue date e il suo stato.

Ogni classe incapsula un concetto del dominio senza includere operazioni non attinenti.

Archivi (ArchivioLibri, ArchivioUtenti, ArchivioPrestitiAttivi, ArchivioCronologiaPrestiti): Coesione Comunicazionale

Gli archivi hanno coesione comunicazionale, perché tutte le operazioni al loro interno lavorano sugli stessi dati:

- collezione di libri,
- insieme di utenti,
- prestiti attivi o storicizzati.

Ricerca, aggiunta, rimozione e salvataggio su file agiscono tutte sulla stessa struttura dati. Non contengono logiche applicative.

Service (LibriService, UtentiService, PrestitiService): Coesione Sequenziale

Nei *Service* è presente una coesione sequenziale, perché le operazioni che essi svolgono devono essere eseguite in un ordine ben preciso, dove l'output di ciascun passo diventa l'input del successivo.

Ogni operazione dipende dal risultato della precedente, e l'intera procedura ha senso solo se eseguita in questa sequenza stabilita.

Controller (LibroViewController, UtentiViewController, PrestitiViewController, HomeViewController): Coesione Logica / Comunicazionale

I controller mostrano una coesione comunicazionale, perché tutte le operazioni lavorano sui dati che arrivano dalla UI e dai service (aggiornamento tabelle, lettura input, gestione feedback).

Presentano anche elementi di coesione logica, perché raccolgono diverse operazioni simili che riguardano l'interazione utente (clic, pulsanti, messaggi).

Non contengono logica applicativa e fungono da mediatori tra View e Service.

4.4.2 Accoppiamento

L'architettura mira a mantenere un accoppiamento debole. Nel sistema compaiono le seguenti tipologie.

Dominio → nessun accoppiamento forte

Le classi *Libro*, *Utente* e *Prestito* non dipendono dai Service, non dipendono dalla UI e non eseguono operazioni di I/O. L'unico accoppiamento presente è quello necessario tra le classi del dominio stesso (ad esempio *Prestito* è associato a *Utente* e *Libro*), sotto forma di normale accoppiamento per dati. In questo modo il dominio rimane indipendente dagli altri livelli dell'applicazione e può essere riutilizzato o esteso senza modificare la logica di presentazione o di persistenza.

Archivi ↔ Dominio: Accoppiamento per Dati

Tra archivi e dominio avviene accoppiamento per dati:

- gli archivi ricevono o restituiscono solo le informazioni necessarie (ISBN, matricola, id prestito);
- non scambiano strutture o oggetti complessi inutili;
- non accedono direttamente agli attributi privati.

Service - Archivi: Accoppiamento per Dati

I service si appoggiano agli archivi, ma:

- non ne controllano la struttura interna,
- usano solo i metodi pubblici,
- ricevono e passano solo i dati necessari.

Questo corrisponde a un accoppiamento per dati.

Controller - Service: Accoppiamento per Controllo

I controller:

- delegano ai service le operazioni logiche,

- non alterano il loro comportamento interno,
- reagiscono ai valori restituiti o alle eccezioni.

Questo rappresenta un accoppiamento per controllo.

View ↔ Controller: Accoppiamento per dati

La *View* comunica esclusivamente con i controller e non ha alcuna dipendenza diretta da service, archivi o logica di business. Lo scambio avviene solo tramite i dati necessari per aggiornare l'interfaccia o per inoltrare l'input dell'utente, configurando quindi un accoppiamento per dati.



5. Modello dinamico

5.1 Introduzione ai diagrammi di sequenza

I diagrammi di sequenza rappresentano uno strumento fondamentale per comprendere la dinamica del sistema. Essi mostrano l'ordine temporale con cui i messaggi vengono scambiati tra gli oggetti durante l'esecuzione di un caso d'uso specifico.

Nel contesto dell'applicazione biblioteca i diagrammi di sequenza evidenziano:

- la separazione netta tra controller, service e archivi, confermando l'adozione del pattern MVC;
- il flusso delle operazioni, dalla ricezione dell'input utente alla persistenza dei dati;
- i punti di validazione e controllo, dove vengono applicate le regole di dominio;
- la gestione delle eccezioni e dei flussi alternativi.

Ogni diagramma è accompagnato da una descrizione testuale che spiega il flusso principale e le possibili deviazioni, fornendo una visione completa del comportamento del sistema.

5.2 Sequenza: Registrazione di un libro

La registrazione di un nuovo libro nel catalogo rappresenta una delle operazioni fondamentali del sistema. Come si constata in Figura 6, il processo coinvolge il LibroViewController, che gestisce l'interfaccia utente, il LibriService, che applica la logica di business, e l'ArchivioLibri, che si occupa della persistenza.

5.2.1 Flusso Principale

1. Il bibliotecario accede alla sezione “Libri” e visualizza la schermata apposita.
2. Il bibliotecario compila i campi obbligatori: titolo, lista degli autori, anno di pubblicazione, ISBN e numero di copie disponibili.
3. Dopo aver premuto il pulsante “Aggiungi nuovo libro”, il LibroViewController intercetta l'evento associato al metodo onAggiungiLibro().
4. Il service effettua una validazione dei dati inseriti, verificando che:
 - tutti i campi obbligatori siano compilati;
 - l'anno sia un valore numerico valido;
 - l'ISBN rispetti il formato previsto;
 - il numero di copie sia un valore intero non negativo.
5. Se la validazione ha esito positivo il LibriService effettua un controllo di unicità interrogando l'ArchivioLibri tramite il metodo ricercaISBN(ISBN).
6. L'ArchivioLibri esegue una ricerca nella propria collezione interna e restituisce il risultato.
7. Se l'ISBN non è presente (la ricerca restituisce NULL), il LibriService crea un'istanza della classe Libro con i dati forniti.
8. Il LibriService procede con l'inserimento invocando aggiungiLibro(libro) sull'ArchivioLibri.
9. L'ArchivioLibri inserisce il nuovo libro nella struttura e conferma l'operazione.

10. Il controllo ritorna al LibroViewController che mostra un messaggio di conferma all'utente.
11. Il nuovo libro rimane memorizzato nell'archivio in memoria e sarà salvato permanentemente su file al momento della chiusura dell'applicazione, quando viene invocato automaticamente il metodo `scriviSuFile()`.

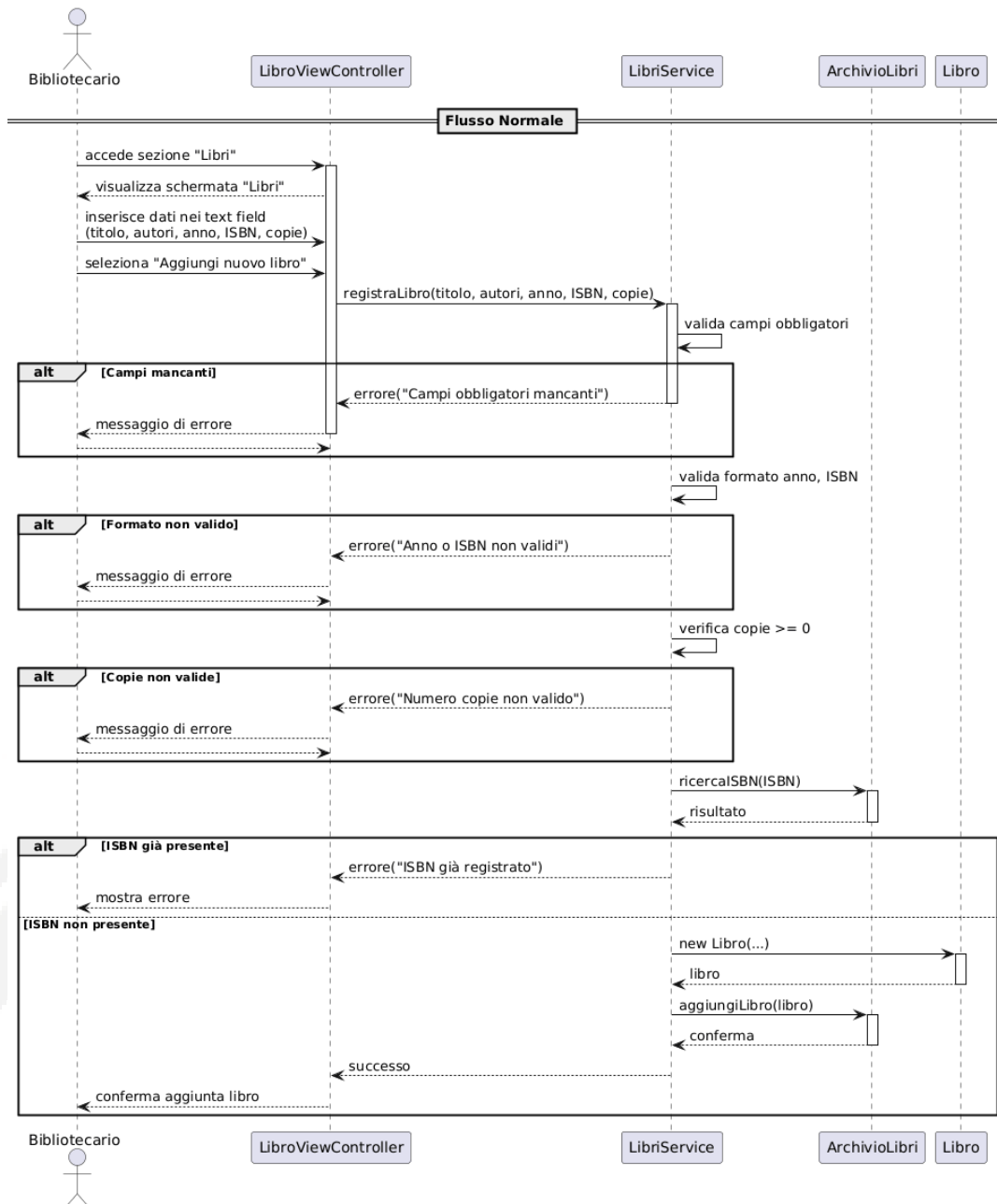


Figura 6 - Diagramma di sequenza: registrazione di un libro

5.2.2 Gestione dei flussi alternativi

Dati mancanti o non validi

Se la validazione locale dei dati fallisce (presenza di campi vuoti, formato anno o ISBN errato, numero di copie negativo), il controller interrompe immediatamente il flusso e mostra un messaggio di errore all'utente. L'operazione non viene propagata al service, evitando elaborazioni inutili. Il bibliotecario può correggere i dati e riprovare l'inserimento.

ISBN già presente in archivio

Se l'ISBN risulta già registrato, il metodo `ricercaISBN()` restituisce un oggetto `Libro` invece di `NULL`. Il `LibriService` rileva la duplicazione e solleva un'eccezione di tipo `duplicatoException`. Il controller intercetta l'eccezione e mostra il messaggio "ISBN già registrato" all'utente. L'esecuzione riprende permettendo al bibliotecario di modificare l'ISBN o annullare l'operazione.

5.3 Sequenza: Registrazione di un prestito

La registrazione di un prestito rappresentata in Figura 7 è l'operazione più complessa del sistema, poiché coinvolge tre archivi distinti (libri, utenti e prestiti) e richiede numerose validazioni per garantire la coerenza del dominio. Il processo mette in relazione un utente con un libro, creando un'entità `Prestito` che attraverserà diverse fasi del proprio ciclo di vita.

5.3.1 Flusso principale

1. Il bibliotecario accede alla sezione "Prestiti" e visualizza l'interfaccia dedicata.
2. Il sistema richiede l'inserimento della matricola dell'utente. Il bibliotecario inserisce il valore e avvia la ricerca.
3. Il `PrestitiViewController` delega la ricerca al `UtentiService` invocando il metodo `cercaUtentePerMatricola(matricola)`.
4. L'`UtentiService` valida che la matricola sia un numero positivo, quindi interroga l'`ArchivioUtenti` tramite `ricercaMatricola(matricola)`.
5. L'`ArchivioUtenti` esegue la ricerca e, se l'utente esiste, restituisce l'oggetto `Utente` corrispondente.
6. I dati dell'utente (nome, cognome, email) vengono mostrati nell'interfaccia per conferma visiva.
7. Analogamente, il bibliotecario inserisce l'ISBN del libro da prestare e avvia la ricerca.
8. Il controller delega al `LibriService`, che interroga l'`ArchivioLibri` tramite `ricercaISBN(ISBN)`.
9. Se il libro esiste, i suoi dati (titolo, autore, copie disponibili) vengono visualizzati.
10. A questo punto, il service verifica che l'utente selezionato non abbia già tre prestiti attivi. Per farlo, interroga l'`ArchivioPrestitiAttivi` con `getPrestitiAttivi()` e conta i prestiti associati all'utente corrente.
11. Se l'utente ha meno di tre prestiti attivi, il bibliotecario seleziona "Registra prestito" e inserisce la data prevista di restituzione nel formato GG/MM/AAAA.
12. Il controller valida il formato della data e verifica che non sia precedente alla data odierna.
13. Il service controlla che il libro abbia almeno una copia disponibile invocando `getCopieDisponibili()` sull'oggetto `Libro`.

14. Se tutte le verifiche hanno esito positivo, Il PrestitiService crea una nuova istanza di Prestito utilizzando il costruttore che accetta utente, libro, data inizio e data fine. Il costruttore genera automaticamente un ID univoco incrementando un contatore statico e imposta lo stato iniziale a ATTIVO.
15. Il service registra il prestito in entrambi gli archivi:
 - invoce aggiungiPrestitoAttivo(prestito) sull'ArchivioPrestitiAttivi per la gestione operativa quotidiana.
 - invoce aggiunguPrestitoCronologia(prestito) sull'ArchivioCronologiaPrestiti per mantenere lo storico completo.
16. Il service aggiorna la disponibilità del libro invocando decrementaCopie() sull'oggetto Libro.
17. Il service conferma il successo dell'operazione al controller.
18. Il PrestitiViewController mostra un messaggio di conferma all'utente: "Prestito registrato con successo".
19. Il nuovo prestito rimane memorizzato in memoria e sarà salvato su file alla chiusura dell'applicazione.





Figura 7 - Diagramma di sequenza: registrazione di un prestito

5.3.2 Gestione dei flussi alternativi

Utente non trovato

Se la matricola inserita non corrisponde ad alcun utente nell'archivio, il metodo `ricercaMatricola` restituisce NULL.

L'`UtentiService` solleva un'eccezione `UtenteNonTrovatoException`, il controller intercetta l'eccezione e mostra il messaggio "Nessun utente trovato" all'utente.

L'operazione termina senza registrazione del prestito e il bibliotecario può riprovare con una matricola diversa.

Libro non trovato

Analogamente, se l'ISBN non esiste nell'archivio, viene sollevata un'eccezione di tipo `LibroNonTrovatoException` e mostrato il messaggio "Nessun libro trovato". L'operazione termina senza ulteriori elaborazioni.

Limite massimo prestiti raggiunto

Se il conteggio dei prestiti attivi dell'utente raggiunge il valore 3, viene sollevata l'eccezione `LimitePrestitoSuperatoException`, il controller mostra il messaggio "Limite massimo di prestiti raggiunto" e impedisce la prosecuzione. Questa verifica avviene prima di richiedere la data di restituzione, evitando inserimenti inutili.

Nessuna copia disponibile

Se il metodo `getCopieDisponibili()` restituisce 0 il controller intercetta l'eccezione `LibroNonDisponibileException` e mostra "Nessuna copia disponibile" e l'operazione non viene registrata.

Questo controllo è fondamentale per mantenere la coerenza tra il catalogo e i prestiti attivi.

Data di restituzione non valida

Se il formato della data non rispetta lo schema GG/MM/AAAA o se la data è precedente a quella odierna, viene mostrato un messaggio di errore appropriato. Il bibliotecario può correggere la data e riprovare.



5.4 Sequenza: Restituzione di un prestito

La restituzione di un libro in Figura 8 conclude il ciclo di vita di un prestito attivo, aggiornando il suo stato e ripristinando la disponibilità del volume nel catalogo. Questa operazione richiede la collaborazione tra più componenti per garantire che tutte le informazioni rimangano allineate.

5.4.1 Flusso principale

1. Il bibliotecario accede alla sezione "Prestiti" e seleziona l'opzione "Prestiti attivi" oppure utilizza la funzione "Ricerca prestito".
2. Se viene selezionata la visualizzazione completa, il `PrestitiViewController` invoca `visualizzaPrestitiAttivi()` sul `PrestitiService`.
3. Il `PrestitiService` interroga l'`ArchivioPrestitiAttivi` con il metodo `getPrestitiAttivi()`.
4. L'archivio restituisce la lista completa dei prestiti in corso, che viene mostrata nella tabella dell'interfaccia.
5. Se invece viene utilizzata la funzione di ricerca specifica (ad esempio per utente e libro), il sistema mostra l'elenco dei prestiti attivi con tutte le informazioni rilevanti: ID del prestito, nome e cognome dell'utente, matricola, titolo del libro, ISBN, data di inizio e data prevista di restituzione.
6. Il bibliotecario seleziona dalla tabella il prestito relativo al libro restituito.
7. Il bibliotecario conferma l'operazione cliccando su "Conferma Restituzione".
8. Il sistema richiede l'inserimento della data effettiva di restituzione.
9. Il Service valida che la data inserita rispetti il formato GG/MM/AAAA.
10. Se la validazione ha esito positivo, il controller delega la chiusura del prestito al `PrestitiService` invocando `eliminaPrestitoAttivo(prestito)`.
11. Il `PrestitiService` recupera la data prevista di restituzione dall'oggetto `Prestito` tramite il metodo `getDataFine()`.
12. Il service confronta la data effettiva di restituzione con la data prevista per determinare se si è verificato un ritardo.
13. Se la data effettiva è successiva alla data prevista, il service aggiorna lo stato del prestito invocando `setStato(RITARDO)` sull'oggetto `Prestito`. Altrimenti lo stato viene impostato a CHIUSO.
14. Il `PrestitiService` rimuove il prestito dall'`ArchivioPrestitiAttivi` invocando `rimuoviPrestitoAttivo(prestito)`.
15. Il service recupera il libro associato al prestito tramite `getLibro()` e invoca `incrementaCopie()` per ripristinare la disponibilità nel catalogo.
16. Il service conferma il successo dell'operazione al controller.
17. Il `PrestitiViewController` mostra il messaggio "Restituzione registrata con successo" all'utente.

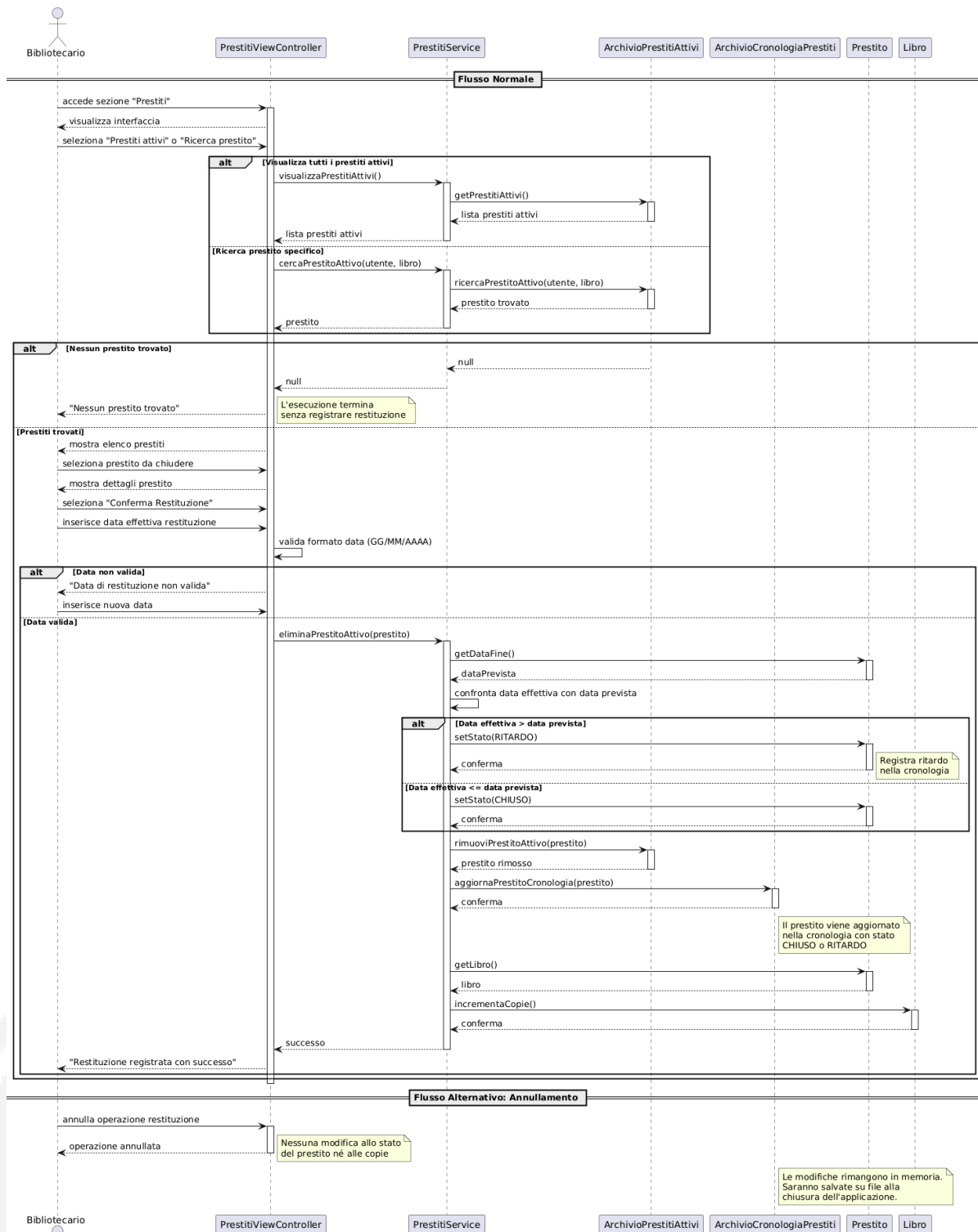


Figura 8 - Diagramma di sequenza: restituzione di un prestito

5.4.2 Gestione dei flussi alternativi

Nessun prestito trovato

Se non esistono prestiti attivi corrispondenti ai criteri di ricerca, o se l'elenco dei prestiti attivi è vuoto, il sistema solleva l'eccezione `PrestitoNonTrovatoException` e mostra il messaggio "Nessun prestito trovato". L'operazione termina immediatamente senza apportare modifiche agli archivi.

Data di restituzione non valida

Se il formato della data inserita non rispetta lo schema GG/MM/AAAA, il controller invoca l'eccezione `ValidazioneException` e mostra un messaggio di errore del tipo "Data di restituzione non valida". L'esecuzione ritorna al punto in cui viene richiesta la data, permettendo al bibliotecario di correggerla e riprovare.

Annullamento dell'operazione

Se il bibliotecario decide di annullare l'operazione prima della conferma finale (ad esempio chiudendo la finestra di dialogo o premendo un pulsante "Annulla"), il sistema non modifica né lo stato del prestito né il numero di copie disponibili. L'esecuzione termina senza impatto sugli archivi.

5.5 Diagramma Macchina a Stati Finiti

Il diagramma a stati finiti in Figura 9 descrive l'evoluzione dell'oggetto `Prestito`, che può assumere tre stati distinti:

- **Attivo:** Stato iniziale. Si attiva alla conferma della registrazione se le precondizioni sono soddisfatte (copie disponibili > 0, limite prestiti utente non superato). Le copie del libro vengono decrementate.
- **In Ritardo:** Transizione automatica. Si verifica se la data odierna supera la data di scadenza prevista senza che il libro sia stato restituito. L'anomalia viene evidenziata nell'interfaccia.
- **Chiuso:** Stato finale. Raggiunto al momento della restituzione del libro (sia da stato Attivo che In Ritardo). Le copie disponibili vengono incrementate e il prestito viene archiviato nello storico.

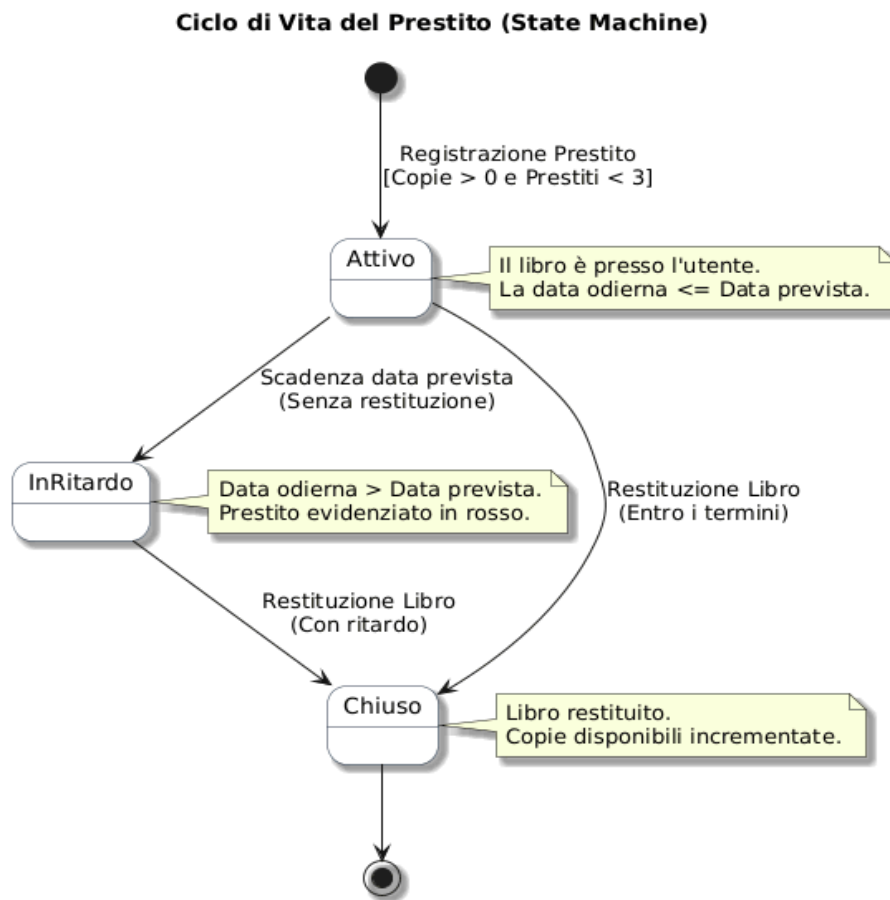


Figura 9 - Diagramma di sequenza: registrazione di un prestito

5.6 Descrizione del flusso dell'applicazione

Il diagramma in Figura 10 mostra il flusso principale dell'applicazione dalla fase di avvio fino alla chiusura. Dopo l'apertura del programma, il bibliotecario viene indirizzato alla schermata *Home*, dalla quale può selezionare una delle tre sezioni disponibili: Libri, Utenti o Prestiti. Una volta scelta la sezione, l'applicazione visualizza l'interfaccia corrispondente e il bibliotecario può eseguire diverse operazioni, come aggiunta, ricerca, modifica o eliminazione dei dati.

Il flusso prevede due controlli ricorrenti: la possibilità di uscire dall'applicazione e la possibilità di cambiare sezione. Se il bibliotecario seleziona "Esci", il programma termina; se invece decide di spostarsi a un altro modulo, il flusso ritorna alla selezione della schermata da visualizzare. In assenza di queste azioni, l'utente rimane nella stessa schermata e può continuare a operare. Questo modello descrive quindi il comportamento interattivo e ciclico dell'interfaccia, garantendo continuità e coerenza nell'esperienza d'uso dell'applicazione.

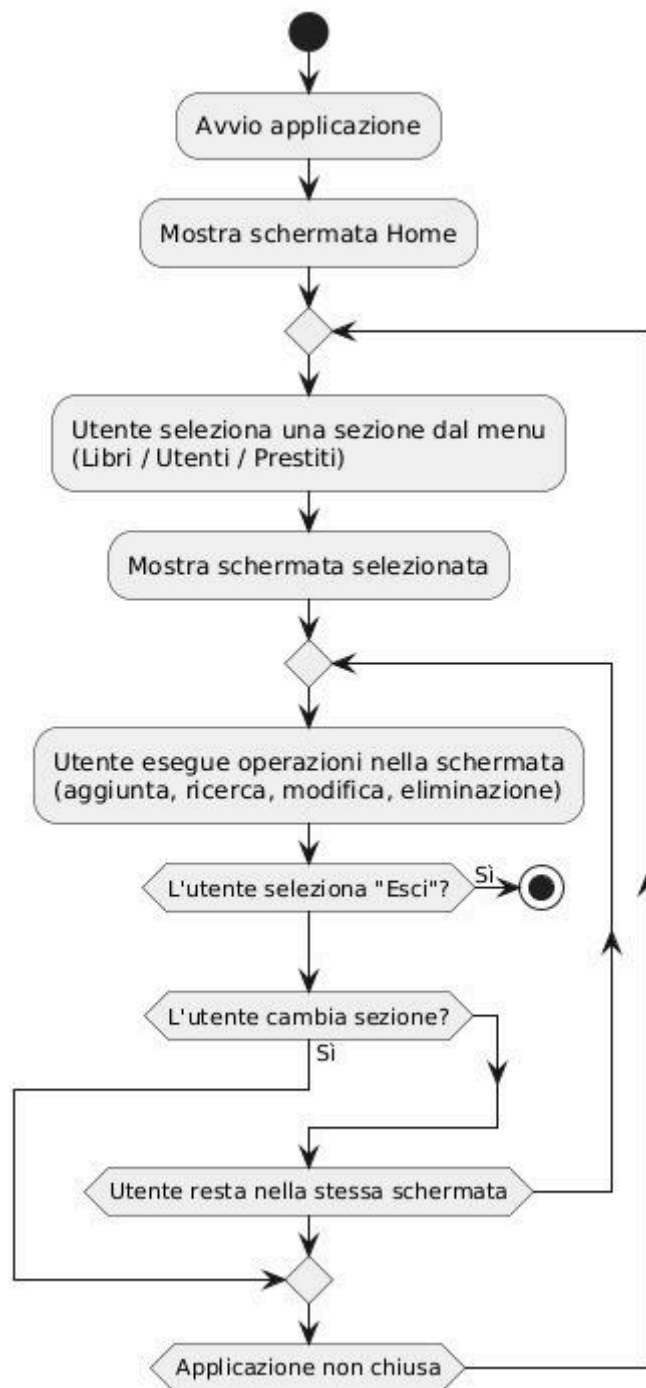


Figura 10 - Diagramma delle attività: gestione

6. Design dell'interfaccia utente

6.1 Linee guida adottate

L'interfaccia utente è stata progettata seguendo criteri di semplicità, coerenza e massima leggibilità. Ogni schermata utilizza lo stesso schema grafico, così da permettere al bibliotecario di riconoscere rapidamente funzioni e pattern di interazione.

La distribuzione dei contenuti privilegia un approccio “a pannelli”: a sinistra si trovano i campi di inserimento e gli strumenti per attivare le operazioni, mentre la parte destra è riservata alla visualizzazione dei dati risultanti da ricerche o aggiornamenti.

Sono stati inoltre adottati:

- layout chiari e non affollati, per ridurre il carico cognitivo;
- colori moderati e coerenti in tutta l'applicazione, con tinte più accese solo per pulsanti rilevanti;
- componenti grafici standard per garantire familiarità;
- messaggi di feedback immediati, così da informare l'utente su esiti, errori o conferme.

6.2 Navigazione tra le schermate

Il sistema utilizza una barra di navigazione orizzontale, sempre presente nella parte superiore dell'interfaccia.

Home Libri Utenti Prestiti

Da questa barra l'utente può accedere direttamente alle quattro sezioni principali: *Home*, *Libri*, *Utenti* e *Prestiti*. Questa scelta progettuale garantisce una navigazione fluida e immediata, riducendo il numero di passaggi necessari per raggiungere una data funzione. La barra rimane invariata in tutte le schermate, fornendo continuità e facilitando l'orientamento dell'utente.

6.3 Mock-up delle schermate

L'applicazione organizza la sua interfaccia in quattro sezioni principali — *Home*, *Libri*, *Utenti* e *Prestiti* — ognuna con una funzione specifica. Questa suddivisione permette di isolare attività diverse e di mantenere ogni area focalizzata sulle operazioni previste dal modulo corrispondente.

6.3.1 Home

La Home (Figura 11) rappresenta il punto di accoglienza dell'utente. È una schermata essenziale, incentrata sul menù di navigazione che consente di raggiungere rapidamente tutte le altre sezioni.

Gli elementi grafici sono volutamente discreti: la parte centrale mostra un'immagine

istituzionale, mentre la barra superiore dà accesso immediato ai moduli *Libri*, *Utenti* e *Prestiti*. La Home non contiene campi operativi, ma funge da snodo principale dell'applicazione.

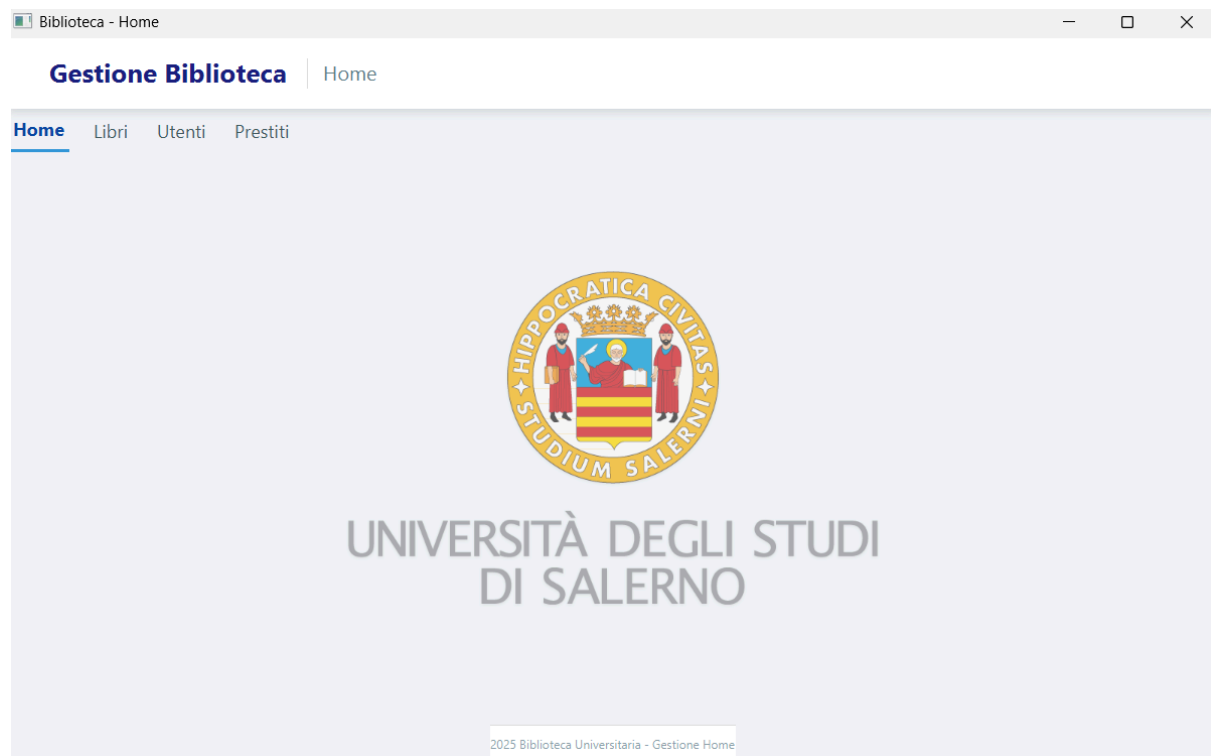


Figura 11 - Home

6.3.2 Schermata Libri

La schermata Libri in Figura 12 consente al bibliotecario di gestire completamente l'archivio dei volumi.

- **Pannello di sinistra**
Qui sono presenti i campi per inserire o ricercare i dati dei libri. Il bottone *Visualizza Tutti i Libri* consente di ottenere l'elenco completo dell'archivio. I pulsanti presenti sotto i campi permettono di eseguire le diverse operazioni, ma risultano attivi solo quando i campi necessari all'operazione selezionata sono stati compilati. In questo modo l'interfaccia previene errori di mancato inserimento e guida il bibliotecario verso un utilizzo corretto delle funzioni disponibili.
- **Pannello di destra**
È dedicato alla visualizzazione dei risultati. La tabella mostra i libri trovati e permette la selezione di un elemento per modificarne i dati o rimuovere l'intero elemento cliccando in seguito sul bottone di eliminazione.

La schermata mantiene la stessa logica visiva adottata negli altri moduli, riducendo i tempi di adattamento del bibliotecario e rendendo immediato il passaggio da una sezione all'altra.

Gestione Biblioteca | Libri

Home **Libri** Utenti Prestiti

Aggiungi / Cerca Libro

Titolo

Autori

ISBN

Numero copie

Anno di pubblica...

Visualizza Tutti i Libri

Risultati Ricerca

Titolo	Autori	ISBN	Copie Totali	Anno Pubbl.
1984	George Orwell	8807900628	4	1949
Cime tempestose	Emily Brontë	8807901726	1	1847
Don Chisciotte	Miguel de Cervantes	8807901533	3	1605
Fahrenheit 451	Ray Bradbury	8807902043	2	1953
I Malavoglia	Giovanni Verga	8807901210	2	1881
I promessi sposi	Alessandro Manzoni	8807900215	2	1840
Il Decameron	Giovanni Boccaccio	8807901428	2	1353
Il barone rampante	Italo Calvino	8807882047	2	1957
Il conte di Montecristo	Alexandre Dumas	8807901611	1	1844
Il fu Mattia Pascal	Luigi Pirandello	8807901105	1	1904
Il giorno della civetta	Leonardo Sciascia	8807900550	1	1961
Il giovane Holden	J. D. Salinger	8807901957	4	1951
Il grande Gatsby	F. Scott Fitzgerald	8807900710	2	1925
Il nome della rosa	Umberto Eco	9788845241459	2	1980
Il vecchio e il mare	Ernest Hemingway	9788807901021	3	1952

2025 Biblioteca Universitaria - Gestione Libri

Figura 12 - Schermata Libri

6.3.3 Schermata Utenti

La schermata Utenti in Figura 13 consente al bibliotecario di gestire completamente l'archivio degli utenti registrati.

- **Pannello di sinistra**
Qui sono presenti i campi per inserire o ricercare i dati dell'utente. Il bottone *Visualizza Utenti* consente di ottenere l'elenco completo dell'archivio. I pulsanti presenti sotto i campi permettono di eseguire le diverse operazioni, ma risultano attivi solo quando i campi necessari all'operazione selezionata sono stati compilati. In questo modo l'interfaccia previene errori di mancati inserimento e guida il bibliotecario verso un utilizzo corretto delle funzioni disponibili
- **Pannello di destra**
È dedicato alla visualizzazione dei risultati. La tabella mostra gli utenti trovati e permette la selezione di un elemento per modificarne i dati o rimuovere l'intero elemento cliccando in seguito sul bottone di eliminazione.

La schermata mantiene la stessa logica visiva adottata negli altri moduli, riducendo i tempi di adattamento del bibliotecario e rendendo immediato il passaggio da una sezione all'altra.

Home Libri **Utenti** Prestiti

Aggiungi / Cerca Utente

Nome

Cognome

Email

Matricola

Visualizza Tutti gli Utenti

Risultati Ricerca

Nome	Cognome	Matricola	Email	Prestiti	Data Restituzione
Anna	Bianchi	2024000002	anna.bianchi@s...	Nessuno	Nessuna
Giuseppe	Caruso	2018000001	giuseppe.carus...	Nessuno	Nessuna
Paolo	Conti	2019000003	paolo.conti@un...	Il fu Mattia Pas...	13/01/2026, 25/...
Michele	Dangelo	2018000003	michele.dangel...	Nessuno	Nessuna
Chiara	DeLuca	2024000006	chiara.deluca@s...	Il conte di Mo...	21/02/2026
Giulia	Esposito	2024000004	giulia.esposito...	Nessuno	Nessuna
Marco	Ferrari	2019000001	marco.ferrari@u...	Nessuno	Nessuna
Simone	Gallo	2024000011	simone.gallo@s...	Nessuno	Nessuna
Alessia	Greco	2024000008	alessia.greco@s...	Nessuno	Nessuna
Federica	Lombardi	2024000010	federica.lombar...	Nessuno	Nessuna
Antonio	Marino	2024000009	antonio.marino...	Il giorno della ...	30/01/2026, 14/...
Sara	Moretti	2019000004	sara.moretti@u...	Nessuno	Nessuna
Ilaria	Palmieri	2018000004	ilaria.palmieri@...	Nessuno	Nessuna
Valentina	Rizzo	2024000012	valentina.rizzo...	Nessuno	Nessuna
Francesco	Romano	2024000005	francesco.roma...	Fahrenheit 451	15/03/2026

2025 Biblioteca Universitaria - Gestione Utenti

Figura 13 - Schermata Utenti

6.3.4 Schermata Prestiti

La sezione Prestiti (Figura 14) integra operazioni più articolate, poiché coinvolge utenti, libri e gestione dello stato dei prestiti.

- **Registrazione di un nuovo prestito:**
L'utente inserisce ISBN del libro, matricola dello studente e data di restituzione. Premendo il pulsante dedicato, il sistema verifica la disponibilità del libro, l'esistenza dell'utente e la validità della data, registrando il prestito in caso positivo.
- **Restituzione di un prestito**
La restituzione avviene selezionando direttamente il prestito dalla visualizzazione dei prestiti attivi e confermando l'operazione tramite il comando Restituisci. Il sistema aggiorna automaticamente lo stato del prestito a Chiuso. Per facilitare l'individuazione del prestito, è possibile effettuare la ricerca per ISBN, per matricola o combinando entrambi i criteri.
- **Visualizzazione**
La tabella inferiore mostra, tramite gli appositi pulsanti *Visualizza Prestiti* e *Cronologia* rispettivamente i prestiti attivi e la cronologia dei prestiti conclusi.

In questa schermata i prestiti in ritardo sono evidenziati graficamente in rosso, rendendoli immediatamente riconoscibili. I pulsanti presenti permettono di eseguire le diverse operazioni, ma risultano attivi solo quando i campi necessari all'operazione selezionata sono

stati compilati. In questo modo l'interfaccia previene errori di mancata inserimento e guida il bibliotecario verso un utilizzo corretto delle funzioni disponibili.

Gestione Biblioteca | Prestiti

Home Libri Utenti **Prestiti**

Ricerca Libro
ISBN

Ricerca Utente
Matricola

Inserisci
Data di Restituzione

Prestiti Registrati

ID	Nome	Cognome	Matricola	Email	Titolo	Autori	ISBN	Anno	Data Inizio	Data Fine	Stato
1	Mario	Rossi	2024000001	mario.rossi@s...	I Malavoglia	Giovanni Verga	8807901...	1881	2025-12-...	2025-12-14	ATTIVO
2	Paolo	Conti	2019000003	paolo.conti@...	Il fu Mattia ...	Luigi Pirandello	8807901...	1904	2025-12-...	2026-01-13	ATTIVO
3	Chiara	DeLuca	2024000006	chiara.deluca...	Il conte di M...	Alexandre Du...	8807901...	1844	2025-12-...	2026-02-21	ATTIVO
4	Elena	Russo	2019000002	elena.russo@...	Se questo Æ...	Primo Levi	8807881...	1947	2025-12-...	2026-01-15	ATTIVO
5	Antonio	Marino	2024000009	antonio.marin...	Il giorno del...	Leonardo Scia...	8807900...	1961	2025-12-...	2026-01-30	ATTIVO

2025 Biblioteca Universitaria - Gestione Prestiti

Figura 14 - Schermata Prestiti

6.4 Gestione dei messaggi

L'applicazione comunica costantemente con l'utente tramite finestre di dialogo (Alert) che forniscono un feedback chiaro e tempestivo.

Si distinguono tre categorie principali:

- **Messaggi di errore (Figura 15)**
Segnalano dati non validi, ricerche senza risultato o operazioni non consentite, bloccano temporaneamente il flusso.
- **Conferme di operazioni**
Utilizzate per azioni sensibili come eliminazioni o modifiche di dati già presenti nell'archivio.
- **Messaggi informativi**
Visualizzati quando un'operazione va a buon fine, come la chiusura di un prestito.

La gestione uniforme di questi messaggi assicura chiarezza ed evita comportamenti ambigui all'interno dell'applicazione.

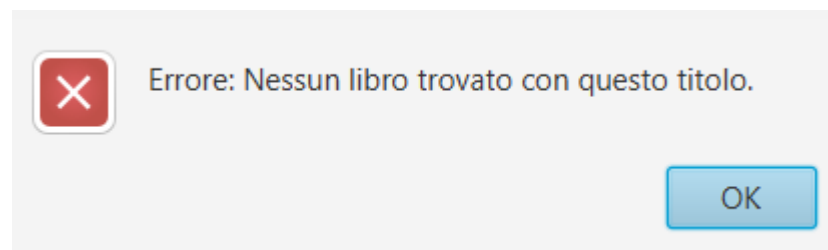


Figura 15 - Gestione dei messaggi

